# Constraint-Aware Minority Oversampling via Relaxed Functional Dependencies [Supplementary Materials]

Mary Cerullo
Liris, CNRS
Lyon, France
mary.cerullo@liris.cnrs.fr

Stefano Cirillo
University of Salerno
Fisciano, Salerno, Italy
scirillo@unisa.it

Gianpaolo Iuliano
LIRIS, CNRS
Lyon, France
gianpaolo.iuliano@cnrs.fr

Giuseppe Polese
University of Salerno
Fisciano, Salerno, Italy
gpolese@unisa.it

Giandomenico Solimando
University of Salerno
Fisciano, Salerno, Italy
gsolimando@unisa.it

## 1 ADDITIONAL ALGORITHMS

In this section, we report the pseudocode of the procedures omitted from the main text: the complete training loop of SyRFDRes (Algorithm 4) and proof of the correctness of VCheck (Proof 1.2).

### 1.1 Training Procedure

Algorithm 4 details the procedure behind the two-phase training schedule (Sec. 4.3). The procedure starts with the initialization of the TabResNet generator, the optimizer with its cosine-restart scheduler, and the rolling buffer (Lines 1–2). Then, it computes the per-attribute normalization statistics $\mathbf{a}$ and $\mathbf{r}$ from $D^+$ and prepares two reference views of the minority partition: the raw tensor $\mathbf{X}_{\text{real}}$ (Line 4), which $\mathcal{L}_{\text{RFD}}$ consumes in original-domain units so that the thresholds in $\Phi$ retain their physical meaning, and its normalized version $\tilde{\mathbf{X}}_{\text{real}}$, which is applied for computing the Silhouette and DBI monitoring statistics. At each iteration, the algorithm applies the sigmoid $\sigma$ to the raw generator output, obtaining the normalized batch $\hat{\mathbf{x}}_n \in [0,1]^m$, and denormalized it into $\hat{\mathbf{x}}_d$ (Lines 10–11): the former feeds the geometric terms, the latter feeds $\mathcal{L}_{\text{MMD}}$ and $\mathcal{L}_{\text{RFD}}$. The algorithm then samples a real mini-batch with replacement, normalizes it, and pushes the generated batch into the buffer $Q$. During the warmup phase, the procedure assembles the objective of Eq. (11) (Lines 16–17); afterwards, it switches to the full objective of Eq. (5) and logs the monitoring statistics over $Q$ and $\tilde{\mathbf{X}}_{\text{real}}$ (Lines 18–22). After the backpropagation of the loss, clipping the gradient norms, and stepping the optimizer and the scheduler, the procedure evaluates the early-stopping criterion (Lines 24–29) only in Phase 2 and only once $Q$ has reached its minimum fill $q_{\min}$, so that the decision to stop rests on stable estimates rather than on single-batch noise. Finally, the procedure returns the generator $G_\theta$.

### 1.2 Proof of the Correctness of VCheck

We report the proof of Lemma 4.1 (Sec. 4.5), which establishes the correctness of VCheck (Algorithm 1): given a finite set $\mathcal{T}$ of tuples satisfying all RFDs in $\Sigma$ pairwise, VCheck$(\mathbf{c}, \mathcal{T}, \Sigma, \Phi)$ = true if and only if $\mathcal{T} \cup \{\mathbf{c}\}$ satisfies all RFDs in $\Sigma$ pairwise.

Proof. Pairwise satisfaction on $\mathcal{T} \cup \{\mathbf{c}\}$ requires checking two types of pairs: (i) $(t, t')$ with $t, t' \in \mathcal{T}$, satisfied by assumption; (ii) $(\mathbf{c}, t)$ and $(t, \mathbf{c})$ for $t \in \mathcal{T}$; by symmetry of viol, it suffices to check $(\mathbf{c}, t)$. By Eq. (13), VCheck$(\mathbf{c}, \mathcal{T}, \Sigma, \Phi)$ = true means invalid$(\mathbf{c})$ = false, i.e.,

$$\neg \bigvee_{(X \to Y) \in \Sigma} \bigvee_{t \in \mathcal{T}} \left[ \bigwedge_{A \in X} |\mathbf{c}[A] - t[A]| \le \phi_A \ \wedge \ |\mathbf{c}[Y] - t[Y]| > \phi_Y \right].$$

By De Morgan's law, this is equivalent to: for every $(X \to Y) \in \Sigma$ and every $t \in \mathcal{T}$,

$$\bigwedge_{A \in X} |\mathbf{c}[A] - t[A]| \le \phi_A \implies |\mathbf{c}[Y] - t[Y]| \le \phi_Y,$$

i.e., viol$(\mathbf{c}, t, \varphi) = 0$ for every $t \in \mathcal{T}$ and every $\varphi \in \Sigma$. Since each step is an equivalence, both directions follow: $\mathbf{c}$ is accepted exactly when no pair involving it violates any RFD in $\Sigma$. □

## 2 ADDITIONAL EXAMPLES

In this section, we illustrate how the two components of SyRFDRes act on generated tuples on the running example of Table 1.

### 2.1 Example associated with the RFD Validator

The following example shows how the validator of Alg. 1 turns the same comparisons into an exact accept/reject decision, discarding a candidate as soon as a single reference tuple introduce a violation.

Example 2.1. *Let us consider the snippet of Page-Blocks in Table 1, the RFD $\varphi$ : length$_{(\le 2)} \to$ height$_{(\le 2)}$, and the minority tuples $t_8, t_9, t_{10}$ as reference set. Let us suppose that the generator produces two candidate tuples: $c_1$ with length = 34 and height = 57, and $c_2$ with length = 30 and height = 29. For $c_2$, all three values are similar to the values for $t_8, t_9, t_{10}$ on both the LHS and RHS of $\varphi$, and $c_2$ is accepted. For $c_1$, the tuple $t_9$ has a similar value for length, i.e., $|34 - 32| = 2 \le 2$, but the corresponding heights differ beyond the threshold, i.e., $|57 - 28| = 29 > 2$, hence* invalid$(c_1)$ *holds and $c_1$ is discarded, showing that a single violating reference suffices.*

### 2.2 Example associated with the SoftRFD Loss

The following example shows how the SoftRFD loss of Eq. (8) assigns a continuous penalty to each generated tuple, distinguishing tuples that actually violate $\varphi$ from those that are compliant or lie close to the thresholds.

Example 2.2. *For instance, let us consider the RFD $\varphi$ : length$_{(\le 2)} \to$ height$_{(\le 2)}$ in Table 1, and suppose the generator produces the tuple $t_{11}^{smo}$, whose distance from $t_4$ is 2 on* length *and 4 on* height. *With*

**Algorithm 4** SyRFDRes Training

**Require:** minority partition $D^+$, RFDs $\Sigma$, thresholds $\Phi$, iterations $T$, warmup $T_w$, batch size $B$, temperature $\tau$, projections $P$, weights $\lambda_{\text{fid}}, \lambda_{\text{rfd}}, \lambda_{\text{comp}}, w_{\text{swd}}, w_{\text{cov}}, w_{\text{div}}$, patience $p_{\max}$, tolerance $\epsilon_{stop}$

Initialise $G_\theta$ (TabResNet), Adam optimiser, cosine-restart scheduler
Initialise rolling buffer $Q$ with capacity $q_{\max}$, min fill $q_{\min}$
$\mathbf{a} \leftarrow \min(D^+)$, $\mathbf{r} \leftarrow \max(D^+) - \mathbf{a}$ ▷ per-attribute min and range
$\mathbf{X}_{\text{real}} \leftarrow D^+$ ▷ raw reference tensor, original domain
$\tilde{\mathbf{X}}_{\text{real}} \leftarrow \text{normalise}(D^+, \mathbf{a}, \mathbf{r})$ ▷ normalized reference, Silhouette/DBI monitoring only
$\ell^* \leftarrow +\infty$, $p \leftarrow 0$
**for** $e = 1$ **to** $T$ **do**
  $\mathbf{Z} \sim \mathcal{N}(0, I_{d_z})^B$
  $\hat{\mathbf{x}}_{\text{raw}} \leftarrow G_\theta(\mathbf{Z})$
  $\hat{\mathbf{x}}_n \leftarrow \sigma(\hat{\mathbf{x}}_{\text{raw}})$ ▷ single sigmoid: output in $[0,1]^m$
  $\hat{\mathbf{x}}_d \leftarrow \hat{\mathbf{x}}_n \odot \mathbf{r} + \mathbf{a}$ ▷ denormalize to original domain
  Sample mini-batch $\mathbf{x}_B \sim D^+$; $\mathbf{x}_n \leftarrow \text{clamp}((\mathbf{x}_B - \mathbf{a})/\mathbf{r}, 0, 1)$
  Push $\hat{\mathbf{x}}_n$ into buffer $Q$
  $\mathcal{L}_{\text{MMD}} \leftarrow \text{MMD}(\hat{\mathbf{x}}_d, \mathbf{x}_B, \mathbf{a}, \mathbf{r})$ ▷ Eq. (7)
  $\mathcal{L}_{\text{RFD}} \leftarrow \text{SoftRFD}(\hat{\mathbf{x}}_d, \mathbf{X}_{\text{real}}, \Sigma, \Phi, \tau)$ ▷ Eq. (8), full raw reference
  **if** $e \le T_w$ **then** ▷ Phase 1 - Warmup
    $\mathcal{L} \leftarrow \lambda_{\text{fid}} \mathcal{L}_{\text{MMD}} + \lambda_{\text{rfd}} \mathcal{L}_{\text{RFD}}$ ▷ Eq. (11)
  **else** ▷ Phase 2 - Refinement
    $\mathcal{L}_{\text{SWD}} \leftarrow \text{SWD}(\hat{\mathbf{x}}_n, \mathbf{x}_n, P)$ ▷ Eq. (9)
    $\mathcal{L}_{\text{cov}} \leftarrow \text{CovLoss}(\hat{\mathbf{x}}_n, \mathbf{x}_n, w_{\text{div}})$ ▷ Eq. (10)
    $\mathcal{L}_{\text{comp}} \leftarrow w_{\text{swd}} \mathcal{L}_{\text{SWD}} + w_{\text{cov}} \mathcal{L}_{\text{cov}}$ ▷ Silhouette/DBI logged using $Q, \tilde{\mathbf{X}}_{\text{real}}$
    $\mathcal{L} \leftarrow \lambda_{\text{fid}} \mathcal{L}_{\text{MMD}} + \lambda_{\text{rfd}} \mathcal{L}_{\text{RFD}} + \lambda_{\text{comp}} \mathcal{L}_{\text{comp}}$ ▷ Eq. (5)
  $\mathcal{L}$.backward(); clip($\nabla_\theta$, 1.0); Adam step; scheduler step
  **if** $e > T_w$ **and** $|Q| \ge q_{\min}$ **then** ▷ early stopping on Phase 2 only
    **if** $\mathcal{L}_{\text{comp}} < \ell^* - \epsilon_{stop}$ **then**
      $\ell^* \leftarrow \mathcal{L}_{\text{comp}}$; $p \leftarrow 0$
    **else**
      $p \leftarrow p + 1$
      **if** $p \ge p_{\max}$ **then break**
**return** $G_\theta$

*$\tau = 0.5$, the pair $(t_{11}^{smo}, t_4)$ contributes $\sigma(0) \cdot \sigma(4) \approx 0.5 \cdot 0.98 \approx 0.49$ to the inner sum of Eq.* (8)*: the LHS condition is triggered and the RHS threshold is exceeded, so the generator receives a strong gradient pushing $t_{11}^{smo}$ out of this configuration. Conversely, the pair $(t_{11}^{smo}, t_2)$, with distance 59 on* length*, contributes $\sigma(-114) \cdot \sigma(\cdot) \approx 0$: pairs that do not trigger the LHS condition are effectively ignored, regardless of their RHS distance.*

# 3 EVALUATION METRICS

Let $R = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n_r)}\}$ be the real minority tuples and $G = \{\hat{\mathbf{x}}^{(1)}, \ldots, \hat{\mathbf{x}}^{(n_g)}\}$ the generated ones, defined over $m$ common numerical attributes. Attribute-wise metrics are computed per attribute $A$ and averaged over the $m$ attributes. Histogram-based metrics discretize each attribute into $b$ equal-width bins over the combined range of $R$ and $G$, with add-one smoothing; $p_A$ and $q_A$ denote the resulting generated and real bin frequencies. Spatial metrics operate on min–max normalized data, with the scaler fitted on $R$ only, generated values clipped to $[0,1]$, and zero-variance attributes excluded. We denote by $\mu_{\cdot,A}$, $s_{\cdot,A}$, and $v_{\cdot,A}$ the mean, standard deviation, and variance of attribute $A$ in the given set, and by $F_{\cdot,A}$ its Cumulative Distribution Function (CDF).

## 3.1 Fidelity Metrics

*Kullback–Leibler divergence [36].* The average divergence of the generated histograms from the real ones ($b = 30$):

$$\text{KL} = \frac{1}{m} \sum_{A=1}^{m} \sum_{i=1}^{b} p_A(i) \, \log \frac{p_A(i)}{q_A(i)}. \tag{15}$$

It penalizes generated mass placed where the real distribution has little support: values close to zero indicate that the generated histograms closely track the real ones, while large values reveal attributes where the generator over-populates regions the real data rarely visits.

*Jensen–Shannon divergence [44].* The symmetrized, bounded counterpart of KL, with $M_A = \frac{1}{2}(p_A + q_A)$:

$$\text{JS} = \frac{1}{m} \sum_{A=1}^{m} \frac{1}{2} D_{\text{KL}}(p_A \,\|\, M_A) + \frac{1}{2} D_{\text{KL}}(q_A \,\|\, M_A). \tag{16}$$

Unlike KL, it is finite even on disjoint supports: values near zero indicate near-identical generated and real histograms, whereas values approaching its upper bound (log 2, attribute-wise) indicate that the two distributions barely overlap.

*Kolmogorov–Smirnov statistic [5, 55].* The maximum gap between the empirical CDFs:

$$\text{KS} = \frac{1}{m} \sum_{A=1}^{m} \sup_x \left| F_{G,A}(x) - F_{R,A}(x) \right|. \tag{17}$$

It captures the worst possible distributional mismatch per attribute: values close to zero mean that the generated and real distributions never deviate much, while values close to one indicate a point where almost none of the generated mass matches the real one.

*$\chi^2$ statistic [2].* The Pearson statistic between generated and real bin counts ($b = 20$), with generated counts rescaled to the real total and the result normalized by $n_g$:

$$\chi^2 = \frac{1}{m} \sum_{A=1}^{m} \frac{1}{n_g} \sum_{i=1}^{b} \frac{\left(\tilde{o}_A(i) - e_A(i)\right)^2}{e_A(i)}, \tag{18}$$

where $e_A$ are the real counts and $\tilde{o}_A$ the rescaled generated counts. Values near zero indicate that generated and real bin counts agree closely, while large values flag bins where the generated frequency deviates sharply from what the real histogram would predict.

*Wasserstein distance [19, 50].* The mean absolute gap between quantile functions, evaluated on a grid of 100 equally spaced levels $u_t \in [0,1]$:

$$\text{WD} = \frac{1}{m} \sum_{A=1}^{m} \frac{1}{100} \sum_{t=1}^{100} \left| F_{R,A}^{-1}(u_t) - F_{G,A}^{-1}(u_t) \right|. \tag{19}$$

Being quantile-based, it remains informative on disjoint supports and is sensitive to tails: values close to zero mean the generated and

real quantile functions nearly coincide at every level, while large values indicate that the generated attribute is systematically shifted or stretched relative to the real one, including in its extremes.

*PCA Wasserstein distance [50].* The metric computes the same quantile distance as WD with the PCA fitted on normalized real data and applied to the generated data:

$$\mathrm{WD}_{\mathrm{PCA}} = \frac{1}{3}\sum_{d=1}^{3}\frac{1}{100}\sum_{t=1}^{100}\left|F^{-1}_{R,d}(u_t) - F^{-1}_{G,d}(u_t)\right|. \tag{20}$$

Unlike WD, which treats each dimension independently and is blind to inter-attribute correlations, $\mathrm{WD}_{\mathrm{PCA}}$ compares the joint distributions along their directions of maximal variance, capturing whether the generated set preserves the correlation structure of the real data rather than just its marginals: values near zero indicate that the dominant modes of joint variation are preserved, while large values reveal that the generated tuples distort the correlations among attributes even if individual marginals look correct.

*Propensity score MSE (pMSE) [56].* A linear propensity model $\hat{\pi}$ is fitted by least squares to discriminate real from generated normalized tuples, with predictions clipped to $[0, 1]$:

$$\mathrm{pMSE} = \frac{1}{n_r + n_g}\sum_{i=1}^{n_r+n_g}\left(\hat{\pi}_i - c\right)^2, \qquad c = \frac{n_g}{n_r + n_g}. \tag{21}$$

If the two sets are indistinguishable, every prediction collapses to the base rate $c$ and the metric approaches zero, meaning no linear model can tell generated and real tuples apart; larger values indicate that real and generated tuples occupy identifiably different regions of the attribute space.

*Maximum Mean Discrepancy [7].* The multi-scale RBF estimate of Eq. (7) with the kernel of Eq. (6), computed on normalized data. It compares the joint distributions through their kernel mean embeddings, capturing discrepancies in all moments: values near zero indicate that generated and real samples are statistically indistinguishable under the kernel embedding, while larger values indicate a growing mismatch between the two distributions as a whole, not just in their marginals.

*Moment differences.* The discrepancies of the first four standardized moments on normalized data:

$$\Delta_\mu = \frac{1}{m}\sum_A \frac{|\mu_{R,A} - \mu_{G,A}|}{s_{R,A}}, \qquad \Delta_v = \frac{1}{m}\sum_A \frac{|v_{R,A} - v_{G,A}|}{v_{R,A}}, \tag{22}$$

$$\Delta_{\mathrm{skew}} = \frac{1}{m}\sum_A |\gamma_{R,A} - \gamma_{G,A}|, \tag{23}$$

where $\gamma$ and $\kappa$ denote skewness and excess kurtosis. They localize the mismatch on specific moments, i.e., location, spread, asymmetry, and tail weight: values near zero mean the generated attribute matches the real one on that specific moment, while large values pinpoint exactly which aspect of the distribution is misrepresented.

## 3.2 Diversity Metrics

*Precision, Recall, Density, and Coverage [37, 46].* Let $r_k(\mathbf{x})$ be the distance from $\mathbf{x}$ to its $k$-th nearest neighbor within its own set ($k = 5$), and $\mathbb{1}[\cdot]$ the indicator function:

$$\mathrm{Prec} = \frac{1}{n_g}\sum_j \mathbb{1}\left[\exists\, i : \|\hat{\mathbf{x}}^{(j)} - \mathbf{x}^{(i)}\| \le r_k(\mathbf{x}^{(i)})\right], \tag{24}$$

$$\mathrm{Rec} = \frac{1}{n_r}\sum_i \mathbb{1}\left[\exists\, j : \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(j)}\| \le r_k(\hat{\mathbf{x}}^{(j)})\right], \tag{25}$$

$$\mathrm{Dens} = \frac{1}{n_g}\sum_j \frac{1}{k}\sum_i \mathbb{1}\left[\|\hat{\mathbf{x}}^{(j)} - \mathbf{x}^{(i)}\| \le r_k(\mathbf{x}^{(i)})\right], \tag{26}$$

$$\mathrm{Cov} = \frac{1}{n_r}\sum_i \mathbb{1}\left[\exists\, j : \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(j)}\| \le r_k(\mathbf{x}^{(i)})\right]. \tag{27}$$

Precision measures the fraction of generated tuples lying on the real manifold; Recall and Coverage measure the fraction of real tuples reached by the generated set (with neighborhoods defined by the generated and the real set, respectively); Density refines Precision by counting how many real neighborhoods contain each generated tuple. For all four, values near one indicate that generated and real samples densely overlap in the corresponding sense (fidelity to the manifold for Precision/Density, support coverage for Recall/Coverage), while values near zero indicate that generated tuples fall outside the real manifold (low Precision/Density) or that large parts of the real support are never reached (low Recall/Coverage).

*Silhouette score.* Computed on $G \cup R$ with set membership as cluster label; for each point, $a$ is the mean intra-cluster distance and $b$ the mean distance to the other cluster:

$$\mathrm{Sil} = \frac{1}{n_r + n_g}\sum_i \frac{b_i - a_i}{\max(a_i, b_i)}. \tag{28}$$

Values near zero indicate that generated and real samples are spatially intermixed—desirable in this context, where separability signals distributional mismatch; values approaching one indicate that the two sets form well-separated clusters, i.e., that generated tuples are easily distinguishable from real ones by their location alone.

*Davies–Bouldin index [16].* On the same two-cluster labeling, with $S_G, S_R$ the mean within-set distances to the respective centroids $\mathbf{c}_G, \mathbf{c}_R$:

$$\mathrm{DBI} = \frac{S_G + S_R}{\|\mathbf{c}_G - \mathbf{c}_R\|_2}. \tag{29}$$

Contrary to its use in clustering, higher is better: a large value means the two sets overlap far more than they separate, i.e., the generated and real centroids are close relative to each set's internal spread; a small value means the two centroids are far apart relative to that spread, indicating that generated and real tuples occupy visibly different regions.

*Intra-compactness.* The average per-attribute variance of the generated set:

$$\bar{I}_{Com} = \frac{1}{m}\sum_{A=1}^{m} v_{G,A}. \tag{30}$$

Note that higher values correspond to a more dispersed generated set. Values close to zero reveal mode collapse, i.e., the generator produces nearly identical tuples; values comparable to the real variance indicate that the generated set spreads out as much as the real data, without necessarily implying that it spreads correctly.

*Mean distance to closest record (mDCR) [66].* The average distance of each generated tuple from its nearest real tuple:

$$\text{mDCR} = \frac{1}{n_g} \sum_{j=1}^{n_g} \min_i \|\hat{\mathbf{x}}^{(j)} - \mathbf{x}^{(i)}\|_2. \tag{31}$$

Values near zero indicate near-duplication of real tuples, i.e., the generator is memorizing rather than generalizing; large values indicate drift off the real support, i.e., generated tuples that resemble no real tuple closely; intermediate values are desirable, as they indicate novel but plausible tuples.

*Quantile-bin coverage.* Each real attribute is partitioned into its decile bins. The metric is the fraction of these bins hit by at least one generated value, following the same support-coverage rationale as [22] but adapted to continuous attributes via quantile binning rather than categorical level counts:

$$\text{Cov}_q = \frac{1}{m} \sum_{A=1}^{m} \frac{\left|\{\, i : \exists\, j,\ \hat{x}_A^{(j)} \in \text{bin}_i \,\}\right|}{\#\text{bins}_A}. \tag{32}$$

It verifies that the generated values populate the full range of each real attribute, including low-density regions: values near 1 mean the generated set reaches every decile of the real distribution, including its sparsely populated tails, while low values mean entire regions of the real attribute ranges.

## 3.3 RFD Compliance Metrics

As shown in Sec. 5.1.4, $S$ denotes the synthetic tuples of the augmented dataset and viol the predicate of Eq. (1); as a protocol note, original tuples replicated verbatim in the augmented files are excluded from $S$, since they are compliant by construction and would inflate the scores of all methods. Tuple-level compliance instantiates Eq. (14) at the two scopes of Definitions 3.3 and 3.4:

$$\text{TC}_{\text{rel}} = \frac{\left|\{\, t \in S \ :\ \text{viol}(t, t', \varphi) = 0 \ \ \forall\, t' \in D^+,\ \varphi \in \Sigma \,\}\right|}{|S|}, \tag{33}$$

$$\text{TC}_{\text{str}} = \frac{\left|\{\, t \in S \ :\ \text{viol}(t, t', \varphi) = 0 \ \ \forall\, t' \in D^+ \cup S \setminus \{t\},\ \varphi \in \Sigma \,\}\right|}{|S|}. \tag{34}$$

Table 1 reports the detailed RFD tuple compliance scores for all methods across the three threshold levels $\varphi \in \{2, 4, 8\}$, expressed as mean $\mu$ and standard deviation $\sigma$ over all datasets. The table complements Figure 3 by providing the exact numerical values underlying the visual comparison, allowing a fine-grained inspection of compliance at each threshold level and validation mode. For each method, the relaxed and strict compliance scores are reported separately, reflecting the two granularities at which dependency preservation is measured: relaxed compliance evaluates each synthetic tuple independently against $D^+$, while strict compliance additionally requires consistency among all synthetic tuples in the augmented set. The standard deviations highlight the dataset-dependent variability of compliance across methods: while SyRFDRes with fixed thresholds and matching validation mode achieves zero variance in strict mode, confirming that perfect compliance is guaranteed by construction, most baselines and LLM-based strategies exhibit high variance, indicating that their compliance is heavily influenced by the relational structure of the specific dataset being augmented.

| Algorithm | $\phi = 2$ | | | | $\phi = 4$ | | | | $\phi = 8$ | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Relaxed | | Strict | | Relaxed | | Strict | | Relaxed | | Strict | |
| | mean | std | mean | std | mean | std | mean | std | mean | std | mean | std |
| *Geometric methods* | | | | | | | | | | | | |
| SMOTE | 0.973 | 0.082 | 0.891 | 0.292 | 0.979 | 0.101 | 0.951 | 0.197 | 0.999 | 0.007 | 0.991 | 0.043 |
| SMOTE-CDNN | 0.970 | 0.089 | 0.891 | 0.287 | 0.980 | 0.094 | 0.954 | 0.196 | 0.999 | 0.006 | 0.991 | 0.045 |
| FSMOTE | 0.932 | 0.140 | 0.905 | 0.194 | 0.978 | 0.100 | 0.971 | 0.136 | 0.998 | 0.008 | 0.996 | 0.021 |
| *Deep generative models* | | | | | | | | | | | | |
| CTABGAN | 0.693 | 0.318 | 0.519 | 0.427 | 0.958 | 0.096 | 0.879 | 0.265 | 0.997 | 0.008 | 0.952 | 0.166 |
| CTABGAN+ | 0.932 | 0.140 | 0.828 | 0.320 | 0.987 | 0.061 | 0.933 | 0.216 | 0.997 | 0.011 | 0.959 | 0.152 |
| DDPM | 0.883 | 0.269 | 0.759 | 0.376 | 0.954 | 0.189 | 0.921 | 0.271 | 0.964 | 0.184 | 0.961 | 0.196 |
| GOGGLE | 0.296 | 0.395 | 0.114 | 0.321 | 0.594 | 0.383 | 0.313 | 0.448 | 0.919 | 0.271 | 0.759 | 0.370 |
| TVAE | 0.709 | 0.288 | 0.514 | 0.424 | 0.900 | 0.170 | 0.849 | 0.240 | 0.990 | 0.044 | 0.956 | 0.149 |
| CasTGAN | 0.691 | 0.333 | 0.582 | 0.365 | 0.873 | 0.260 | 0.801 | 0.354 | 0.976 | 0.072 | 0.902 | 0.285 |
| CUTS | 0.927 | 0.155 | 0.725 | 0.397 | 0.987 | 0.061 | 0.879 | 0.323 | 0.999 | 0.004 | 0.923 | 0.272 |
| PreFair | 0.876 | 0.240 | 0.731 | 0.380 | 0.966 | 0.120 | 0.921 | 0.222 | 0.998 | 0.008 | 0.968 | 0.114 |
| TabFairGAN | 0.866 | 0.296 | 0.789 | 0.372 | 0.940 | 0.213 | 0.903 | 0.283 | 0.966 | 0.172 | 0.962 | 0.193 |
| TabularARGN | 0.951 | 0.153 | 0.905 | 0.269 | 0.972 | 0.101 | 0.913 | 0.259 | 0.988 | 0.049 | 0.950 | 0.193 |
| *LLM-based — Devstral-Small-24B* | | | | | | | | | | | | |
| Dev/conf | 0.900 | 0.216 | 0.800 | 0.336 | 0.959 | 0.155 | 0.819 | 0.360 | 0.959 | 0.161 | 0.946 | 0.201 |
| Dev/distr | 0.865 | 0.262 | 0.718 | 0.408 | 0.924 | 0.228 | 0.809 | 0.365 | 0.933 | 0.229 | 0.885 | 0.320 |
| Dev/dtree | 0.840 | 0.271 | 0.657 | 0.419 | 0.910 | 0.243 | 0.818 | 0.362 | 0.915 | 0.253 | 0.884 | 0.325 |
| Dev/ens | 0.795 | 0.285 | 0.474 | 0.439 | 0.899 | 0.227 | 0.678 | 0.421 | 0.918 | 0.233 | 0.833 | 0.368 |
| Dev/hier | 0.921 | 0.156 | 0.774 | 0.362 | 0.958 | 0.133 | 0.841 | 0.326 | 0.952 | 0.168 | 0.912 | 0.274 |
| Dev/react | 0.955 | 0.090 | 0.792 | 0.347 | 0.974 | 0.066 | 0.836 | 0.326 | 0.992 | 0.033 | 0.922 | 0.272 |
| Dev/schema | 0.927 | 0.150 | 0.798 | 0.313 | 0.949 | 0.139 | 0.826 | 0.339 | 0.962 | 0.138 | 0.944 | 0.210 |
| Dev/self | 0.854 | 0.299 | 0.760 | 0.383 | 0.911 | 0.233 | 0.776 | 0.394 | 0.918 | 0.237 | 0.846 | 0.368 |
| *LLM-based — Gemma-3-12B* | | | | | | | | | | | | |
| G3/conf | 0.856 | 0.295 | 0.605 | 0.438 | 0.894 | 0.234 | 0.721 | 0.437 | 0.915 | 0.224 | 0.795 | 0.400 |
| G3/distr | 0.838 | 0.307 | 0.665 | 0.431 | 0.881 | 0.266 | 0.766 | 0.417 | 0.906 | 0.248 | 0.824 | 0.375 |
| G3/dtree | 0.879 | 0.238 | 0.555 | 0.470 | 0.954 | 0.095 | 0.769 | 0.403 | 0.971 | 0.081 | 0.875 | 0.328 |
| G3/ens | 0.861 | 0.187 | 0.390 | 0.412 | 0.928 | 0.166 | 0.587 | 0.450 | 0.940 | 0.171 | 0.798 | 0.382 |
| G3/hier | 0.854 | 0.288 | 0.585 | 0.462 | 0.897 | 0.225 | 0.739 | 0.419 | 0.914 | 0.221 | 0.790 | 0.403 |
| G3/react | 0.857 | 0.279 | 0.592 | 0.442 | 0.909 | 0.226 | 0.790 | 0.374 | 0.920 | 0.225 | 0.839 | 0.360 |
| G3/schema | 0.840 | 0.302 | 0.542 | 0.448 | 0.896 | 0.244 | 0.764 | 0.412 | 0.905 | 0.238 | 0.790 | 0.404 |
| G3/self | 0.862 | 0.291 | 0.595 | 0.427 | 0.929 | 0.205 | 0.763 | 0.409 | 0.931 | 0.215 | 0.793 | 0.398 |
| *LLM-based — Gemma-4-31B* | | | | | | | | | | | | |
| G4/conf | 0.914 | 0.226 | 0.769 | 0.403 | 0.935 | 0.211 | 0.818 | 0.352 | 0.958 | 0.195 | 0.918 | 0.271 |
| G4/distr | 0.919 | 0.195 | 0.764 | 0.381 | 0.937 | 0.177 | 0.842 | 0.340 | 0.960 | 0.136 | 0.913 | 0.272 |
| G4/dtree | 0.876 | 0.239 | 0.748 | 0.393 | 0.895 | 0.246 | 0.785 | 0.373 | 0.918 | 0.236 | 0.847 | 0.365 |
| G4/ens | 0.886 | 0.221 | 0.486 | 0.453 | 0.917 | 0.200 | 0.712 | 0.419 | 0.942 | 0.189 | 0.875 | 0.326 |
| G4/hier | 0.956 | 0.100 | 0.750 | 0.392 | 0.974 | 0.082 | 0.828 | 0.344 | 0.992 | 0.028 | 0.917 | 0.271 |
| G4/react | 0.955 | 0.132 | 0.834 | 0.353 | 0.980 | 0.083 | 0.857 | 0.309 | 0.996 | 0.015 | 0.960 | 0.192 |
| G4/schema | 0.932 | 0.144 | 0.742 | 0.411 | 0.954 | 0.125 | 0.808 | 0.360 | 0.988 | 0.039 | 0.916 | 0.266 |
| G4/self | 0.960 | 0.096 | 0.780 | 0.375 | 0.977 | 0.082 | 0.841 | 0.330 | 0.993 | 0.026 | 0.939 | 0.216 |
| *LLM-based — GPT-OSS-120B* | | | | | | | | | | | | |
| GPT120/conf | 0.839 | 0.293 | 0.712 | 0.407 | 0.926 | 0.211 | 0.775 | 0.386 | 0.947 | 0.190 | 0.878 | 0.324 |
| GPT120/distr | 0.815 | 0.307 | 0.673 | 0.433 | 0.904 | 0.255 | 0.803 | 0.365 | 0.932 | 0.237 | 0.921 | 0.271 |
| GPT120/dtree | 0.806 | 0.291 | 0.493 | 0.426 | 0.888 | 0.223 | 0.655 | 0.427 | 0.929 | 0.214 | 0.791 | 0.402 |
| GPT120/ens | 0.780 | 0.303 | 0.388 | 0.426 | 0.886 | 0.224 | 0.531 | 0.449 | 0.911 | 0.235 | 0.725 | 0.450 |
| GPT120/hier | 0.854 | 0.270 | 0.697 | 0.403 | 0.910 | 0.251 | 0.825 | 0.331 | 0.930 | 0.236 | 0.891 | 0.308 |
| GPT120/react | 0.802 | 0.330 | 0.599 | 0.457 | 0.911 | 0.218 | 0.772 | 0.367 | 0.937 | 0.206 | 0.841 | 0.367 |
| GPT120/schema | 0.800 | 0.323 | 0.604 | 0.446 | 0.898 | 0.256 | 0.786 | 0.373 | 0.929 | 0.237 | 0.842 | 0.367 |
| GPT120/self | 0.809 | 0.312 | 0.668 | 0.408 | 0.910 | 0.218 | 0.797 | 0.339 | 0.941 | 0.201 | 0.883 | 0.325 |
| *LLM-based — GPT-OSS-20B* | | | | | | | | | | | | |
| GPT20/conf | 0.808 | 0.299 | 0.552 | 0.438 | 0.921 | 0.185 | 0.703 | 0.428 | 0.937 | 0.184 | 0.807 | 0.401 |
| GPT20/distr | 0.848 | 0.257 | 0.557 | 0.463 | 0.932 | 0.183 | 0.698 | 0.453 | 0.950 | 0.166 | 0.770 | 0.424 |
| GPT20/dtree | 0.793 | 0.273 | 0.327 | 0.413 | 0.890 | 0.236 | 0.634 | 0.448 | 0.933 | 0.180 | 0.757 | 0.427 |
| GPT20/ens | 0.772 | 0.308 | 0.381 | 0.409 | 0.889 | 0.244 | 0.611 | 0.447 | 0.913 | 0.233 | 0.758 | 0.427 |
| GPT20/hier | 0.816 | 0.304 | 0.496 | 0.454 | 0.907 | 0.205 | 0.614 | 0.446 | 0.931 | 0.197 | 0.795 | 0.400 |
| GPT20/react | 0.807 | 0.296 | 0.421 | 0.430 | 0.907 | 0.199 | 0.632 | 0.446 | 0.931 | 0.195 | 0.806 | 0.401 |
| GPT20/schema | 0.793 | 0.299 | 0.502 | 0.451 | 0.892 | 0.233 | 0.672 | 0.451 | 0.927 | 0.207 | 0.760 | 0.426 |
| GPT20/self | 0.822 | 0.279 | 0.496 | 0.461 | 0.920 | 0.164 | 0.630 | 0.451 | 0.951 | 0.152 | 0.726 | 0.450 |
| *LLM-based — Zero-shot* | | | | | | | | | | | | |
| DeepSeek | 0.957 | 0.126 | 0.888 | 0.277 | 0.976 | 0.097 | 0.927 | 0.256 | 0.972 | 0.126 | 0.940 | 0.215 |
| LLaMA | 0.663 | 0.375 | 0.298 | 0.433 | 0.730 | 0.349 | 0.432 | 0.466 | 0.806 | 0.334 | 0.576 | 0.489 |
| *SyRFDRes* | | | | | | | | | | | | |
| SyRFDRes $\varphi$=2/*rlx* | **1.000** | 0.000 | 0.919 | 0.240 | 0.978 | 0.111 | 0.938 | 0.218 | 0.999 | 0.003 | 0.975 | 0.126 |
| SyRFDRes $\varphi$=2/*str* | **1.000** | 0.000 | **1.000** | 0.000 | 0.983 | 0.084 | 0.945 | 0.210 | **1.000** | 0.002 | 0.980 | 0.102 |
| SyRFDRes $\varphi$=4/*rlx* | 0.951 | 0.138 | 0.884 | 0.297 | **1.000** | 0.000 | 0.942 | 0.212 | 0.999 | 0.004 | 0.974 | 0.131 |
| SyRFDRes $\varphi$=4/*str* | 0.953 | 0.130 | 0.887 | 0.286 | **1.000** | 0.000 | **1.000** | 0.000 | 0.999 | 0.005 | 0.984 | 0.081 |
| SyRFDRes $\varphi$=8/*rlx* | 0.963 | 0.129 | 0.901 | 0.243 | 0.982 | 0.089 | 0.945 | 0.205 | **1.000** | 0.000 | 0.979 | 0.105 |
| SyRFDRes $\varphi$=8/*str* | 0.955 | 0.123 | 0.880 | 0.300 | 0.980 | 0.102 | 0.953 | 0.198 | **1.000** | 0.000 | **1.000** | 0.000 |
| SyRFDRes *var*/*rlx* | **1.000** | 0.000 | **1.000** | 0.000 | 0.667 | 0.488 | 0.600 | 0.507 | **1.000** | 0.000 | **1.000** | 0.000 |
| SyRFDRes *var*/*str* | **1.000** | 0.000 | **1.000** | 0.000 | 0.615 | 0.506 | 0.615 | 0.506 | **1.000** | 0.000 | **1.000** | 0.000 |

$\varphi$: similarity threshold used for RFD$_c$ discovery; *rlx*: relaxed validation mode; *str*: strict validation mode; *var*: variable per-attribute threshold inferred by Domino [12]. *mean*: mean compliance; *std*: standard deviation.

**Table 1: RFD compliance on both relaxed and strict modes.**

# 4 LLM-BASED GENERATION STRATEGIES

We evaluate LLMs as minority-class generators using eight prompting strategies of increasing reasoning complexity.

*Common protocol.* Every prompt shares the same skeleton, composed of: *i)* a role instruction framing the model as a tabular data augmentation expert and stating the target minority class; *ii)* schema constraints, listing for each numerical attribute its admissible range, mean, and standard deviation, and for each categorical attribute its admissible values (the ten most frequent levels, five of which are shown in the prompt); *iii)* distribution guidance, reporting the interquartile statistics (Q25, median, Q75) of each numerical attribute; *iv)* five real minority tuples serving as few-shot examples; and *v)* a strict output-format footer requiring a bare CSV with the exact attribute header, no markdown, and no additional text. Tuples are requested in batches of 10 until the minority class matches the majority count. Each raw response is parsed by a format-tolerant CSV extractor, which strips code fences, locates the header line, and attempts multiple separators (comma, semicolon, tab, pipe); the minority label is then enforced on all parsed rows. Statistics are computed once per dataset and reused across batches.

*Practical details.* Models are queried with temperature 0.7; all other sampling parameters, such as top-$p$, top-$k$, or maximum output tokens, are left at their provider defaults, and no fixed random seed is used, so individual runs are not bit-exact reproducible, though aggregate statistics over the 35 benchmarks are stable. Each request is retried up to 3 times with linear backoff, i.e., 5$s$, 10$s$, and 15$s$, and a 120-second timeout per attempt; a batch that still fails after all retries, i.e., empty response, or a response from which no CSV can be extracted, contributes no rows for that round, so a strategy occasionally finishes with fewer synthetic tuples than the target diff for datasets on which the model struggles to comply with the output format, this is reflected in the raw generation logs released in the repository. Duplicate or near-verbatim copies of the few-shot examples are not filtered at generation time and count toward the generation budget as any other row, to avoid affecting compliance with tuples that are duplicates of the real data.

*Prompting strategies.* Every prompt is built by concatenating a shared header (Template 1), a strategy-specific instruction block, and a shared footer (Template 2). Placeholders in braces are filled at runtime: $n$ is the number of rows requested in the batch, *minority_class* (*mc*) the minority label, *profile.schema_constraints* and *profile.distribution_summary* the per-attribute ranges, admissible values, and distribution statistics, e.g., min, max, or quartile values, computed on the real minority partition, *examples_csv* five real minority tuples, and *cols* the exact output header.

**Template 1: Shared header**

```
You are an expert in data augmentation for tabular datasets. You must generate
exactly {n} new synthetic rows belonging ONLY to the minority class: '{minor-
ity_class}'.
{profile.schema_constraints}
{profile.distribution_summary}
Real examples of the minority class (few-shot): {examples_csv}
```

**Template 2: Shared footer**

```
Respond ONLY with a valid CSV (comma-separated).
Exact header on the first row: {cols}
No additional text, no markdown block, no empty lines.
```

On top of the shared skeleton, each strategy adds a distinct instruction block:

- *Schema Constraints*: mandates strict adherence to the declared ranges and admissible categorical values, forbidding missing fields; it represents the minimal, constraint-only baseline. Further details about the prompt have been provided in the Template 3.

**Template 3: Schema Constraints**

```
[Shared header, Template 1]
Strictly respect the schema constraints:
    - No numeric value outside the indicated ranges.
    - Only allowed values for categorical columns.
    - No NULL or missing fields.
    - Generate exactly {n} rows.
[Shared footer, Template 2]
```

- *Distribution Guidance*: requires the generated values to reflect the observed interquartile ranges, to preserve the inter-attribute correlations visible in the examples, and to avoid verbatim copies. Further details about the prompt were provided in the Template 4.

**Template 4: Distribution Guidance**

```
[Shared header, Template 1]
Follow the distribution guidance to produce realistic data:
    - Numeric values must reflect the observed distribution (Q25–Q75).
    - Introduce realistic variability: do not copy the examples identically.
    - Preserve the correlations between columns present in the examples.
Generate {n} varied but statistically coherent rows.
[Shared footer, Template 2]
```

- *Hierarchical CoT*: guides the model through four explicit reasoning levels before emitting the CSV—schema structure, per-attribute distributions, inter-attribute consistency, and generation. Further details about the prompt were provided in the Template 5.

**Template 5: Hierarchical CoT**

```
[Shared header, Template 1]
Follow these reasoning levels BEFORE writing the CSV:
LEVEL 1 - STRUCTURE
    Which columns are numeric? Which are categorical?
    Are there obvious correlations between columns in the examples?
LEVEL 2 - DISTRIBUTION
    For each numeric column, what is the plausible range?
    For categorical columns, what are the dominant values?
LEVEL 3 - INTERNAL CONSISTENCY
    Do columns influence each other? (e.g. age ↔ income, hours ↔ output)
    The new rows must respect these relationships.
LEVEL 4 - GENERATION
    Now generate the {n} examples applying all constraints identified above.
[Shared footer, Template 2]
```

- *Self-Consistency CoT*: instructs the model to draft the tuples, verify their means and variances against the examples, inspect each row for impossible or contradictory combinations, and emit the corrected CSV only afterwards. Further details about the prompt have been provided in the Template 6.

**Template 6: Self-Consistency CoT**

```
[Shared header, Template 1]
Use this self-verification process before producing the final CSV:
STEP 1 - First draft
    Generate {n} synthetic rows respecting schema and distribution.
STEP 2 - Statistical check
    Are the means and variances of the generated rows consistent with the
examples?
    If not, correct the anomalous values.
STEP 3 - Semantic check
    Does each row make sense? Are there impossible or contradictory combi-
nations?
    If so, fix them.
STEP 4 - Final output
    Write the definitive CSV only after applying the corrections.
[Shared footer, Template 2]
```

- *ReAct CoT*: decomposes generation into interleaved reasoning and acting steps that extract per-attribute statistics, categorical constraints, and the strongest correlations from the examples

before producing the output. Further details about the prompt have been provided in the Template 7.

Template 7: ReAct CoT

*[Shared header, Template 1]*
Apply the ReAct pattern (Reasoning + Acting):
[Thought 1] What are the key statistical characteristics of class '{mc}'?
    [Action 1] Compute the mean and range for each numeric column from the examples.
    [Obs 1] Note the values.
[Thought 2] Are there important categorical constraints?
    [Action 2] List the allowed values for each categorical column.
    [Obs 2] Note the constraints.
[Thought 3] How do the columns correlate with each other?
    [Action 3] Identify the 2–3 strongest correlations from the examples.
    [Obs 3] Note the correlations.
[Thought 4] Generate {n} rows respecting everything above.
    [Action 4] Write the final CSV.
*[Shared footer, Template 2]*

- *Ensemble CoT*: rotates the batches over three perspectives: statistical fidelity, semantic plausibility, and diversity, each instructing the model to prioritize a different quality dimension, so that the final set aggregates complementary viewpoints. Further details about the prompt have been provided in the Template 8.

Template 8: Ensemble CoT

*[Shared header, Template 1]*
Active perspective: {perspective_name}
{perspective_focus}
Reason from this perspective:
1. What must you guarantee as a priority from this perspective?
2. Which columns require more attention?
3. Generate {n} rows optimized according to this perspective.
**Perspectives (one per generation round):**
    STATISTICAL: Priority: statistical fidelity - mean, std and distribution must match.
    SEMANTIC: Priority: semantic plausibility - every row must be realistic within the domain.
    DIVERSITY: Priority: diversity - cover edge cases, rare cases, extreme but plausible values.
*[Shared footer, Template 2]*

- *Decision-Tree CoT*: asks the model to build a mental decision tree rooted at the minority class and branching on the median of a pivot attribute, then to generate tuples evenly across branches to increase variety. Further details about the prompt have been provided in the Template 9.

Template 9: Decision-Tree CoT

*[Shared header, Template 1]*
Mentally build a decision tree and use it to generate:
ROOT → class = '{mc}'
NODE [{pivot_attribute}]
    Branch A: {pivot_attribute} < median
        → which other features tend to be low? which categorical values dominate?
    Branch B: {pivot_attribute} ≥ median
        → which other features tend to be high? which categorical values dominate?
LEAVES → generate examples following the paths.
    Distribute evenly between Branch A and Branch B to maximize variety.
Generate {n} rows in total (balanced between the branches).
*[Shared footer, Template 2]*

- *Confidence CoT*: asks the model to score each candidate row on range adherence, categorical coherence, and semantic plausibility, and to retain only rows above a fixed confidence threshold. Further details about the prompt were provided in the Template 10.

Template 10: Confidence CoT

*[Shared header, Template 1]*
For each row, internally apply this confidence filter:
    Score = 0.4× (numeric range respected)
    + 0.3× (categorical consistency)
    + 0.3× (semantic plausibility)
    If score < 0.7 → discard and regenerate the row.
    Include in the final CSV ONLY rows with score ≥ 0.7.
Generate at least {n} valid rows (the number of discarded rows does not count).
The score column must NOT appear in the CSV - it is only an internal filter.
*[Shared footer, Template 2]*

***Models.*** The eight strategies are run on five models, Devstral-Small 24B, Gemma-3 12B, Gemma-4 31B, GPT-OSS 20B, GPT-OSS 120B, accessed through the Ollama Cloud platform[1], each queried independently with the retry and timeout policy described above; no fallback across models is used. DeepSeek 3 and Llama 3 are additionally evaluated in a plain zero-shot setting, receiving only the shared skeleton without strategy-specific instructions. The verbatim prompt templates are available in our repository.

## 5 FULL FIDELITY AND DIVERSITY EVALUATION

Table 2 reports mean values of all fidelity and diversity metrics averaged across the benchmark datasets shown in Table 2.

### 5.1 Fidelity results

*KL and JS divergence.* KL and JS measure how closely the per-attribute histograms of the generated data match those of the real minority class; both equal zero for identical distributions and grow with increasing discrepancy, with JS bounded in $[0, 1]$. Among the worst-performing models, GOGGLE achieves a KL of 0.567 and CasTGAN of 0.356, suggesting these algorithms tend to overfit the majority structure when trained on heavily imbalanced data. Regarding geometric methods, SMOTE, SMOTE-CDNN, and FSMOTE achieve KL values between 0.260 and 0.317 and JS values around 0.059 and 0.074, indicating that they well preserve the marginal attribute distributions of the minority class by generating synthetic samples through local interpolation between existing minority instances. Among the deep generative models, TabularARGN and TVAE are the most competitive algorithms, reaching KL values of 0.282 and 0.280, respectively. SyRFDRes with fixed thresholds outperform all baselines, with $\phi \in \{4, 8\}$ on *str* mode, obtaining KL values of 0.247 and 0.248, and JS values of 0.060.

*KS and $\chi^2$ statistics.* KS captures the worst-case gap between Cumulative Distribution Functions (CDFs) averaged over attributes, while $\chi^2$ quantifies bin-level histogram discrepancy normalized by the number of generated tuples. When these scores are equal to zero means that the distributions are perfectly matching. As we can see from the results, GOGGLE and CasTGAN are the algorithms with the worst performance, with KS of 0.385 and 0.257 and $\chi^2$ of 0.539 and 0.238, respectively. Instead, TabularARGN achieves the best KS of 0.175 but with a $\chi^2$ of 0.090, followed by SyRFDRes with KS values ranging from 0.181 to 0.186. However, SyRFDRes achieves better $\chi^2$ scores than all baselines, with all fixed-threshold configurations, with a score ranging from 0.0747 to 0.0777.

*Wasserstein distance and PCA Wasserstein distance.* WD measures the distance between the real and generated distributions for each

[1] https://ollama.com/search?c=cloud

| Algorithm | Diversity | | | | | | | | | | Fidelity | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Silh $\overset{0}{\rightarrow}$ | DBI[16]↑ | $\mathcal{I}_{Com}$ | Prec[37]↑ | Rec[37]↑ | Dens[46]↑ | Cov[46]↑ | mDCR[66] | $\text{Cov}_q$[22]↑ | KL[36]↓ | JS[44]↓ | KS[55]↓ | $\chi^2$[2]↓ | WD[50]↓ | pMSE[56]↓ | MMD[7]↓ | $\text{WD}_{\text{PCA}}$[50]↓ | $\Delta_\mu$ ↓ | $\Delta_v$ ↓ | $\Delta_{\text{skew}}$ ↓ |
| CasTGAN [65] | $0.1041_{\pm 0.115}$ | $3.8963_{\pm 2.355}$ | $0.0828_{\pm 0.033}$ | $0.7562_{\pm 0.238}$ | $1.0000_{\pm 0.0}$ | $0.6018_{\pm 0.329}$ | $1.0000_{\pm 0.0}$ | $0.6984_{\pm 0.488}$ | $0.9870_{\pm 0.022}$ | $0.3559_{\pm 0.238}$ | $0.0756_{\pm 0.046}$ | $0.2573_{\pm 0.151}$ | $0.2376_{\pm 0.229}$ | $141.0337_{\pm 712.248}$ | $0.0164_{\pm 0.012}$ | $0.5708_{\pm 0.513}$ | $0.2160_{\pm 0.104}$ | $0.6597_{\pm 0.503}$ | $1.8023_{\pm 5.422}$ | $1.9083_{\pm 2.254}$ |
| CTAB-GAN [66] | $0.0302_{\pm 0.070}$ | $4.9677_{\pm 2.442}$ | $0.0865_{\pm 0.031}$ | $0.7816_{\pm 0.217}$ | $1.0000_{\pm 0.0}$ | $0.7343_{\pm 0.374}$ | $1.0000_{\pm 0.0}$ | $0.7032_{\pm 0.440}$ | $0.9960_{\pm 0.008}$ | $0.3145_{\pm 0.148}$ | $0.0678_{\pm 0.033}$ | $0.3026_{\pm 0.094}$ | $0.1284_{\pm 0.087}$ | $64.2612_{\pm 307.457}$ | $0.0062_{\pm 0.007}$ | $0.3213_{\pm 0.232}$ | $0.1672_{\pm 0.077}$ | $0.5278_{\pm 0.373}$ | $1.6560_{\pm 4.276}$ | $1.5505_{\pm 2.180}$ |
| CTAB-GAN+ [67] | $0.0461_{\pm 0.071}$ | $4.9735_{\pm 2.162}$ | $0.0760_{\pm 0.028}$ | $0.7918_{\pm 0.209}$ | $1.0000_{\pm 0.0}$ | $0.8014_{\pm 0.419}$ | $1.0000_{\pm 0.0}$ | $0.6149_{\pm 0.387}$ | $0.9974_{\pm 0.006}$ | $0.3074_{\pm 0.153}$ | $0.0643_{\pm 0.032}$ | $0.2415_{\pm 0.089}$ | $0.1347_{\pm 0.134}$ | $73.9745_{\pm 374.635}$ | $0.0056_{\pm 0.006}$ | $0.2750_{\pm 0.204}$ | $0.1691_{\pm 0.077}$ | $0.4434_{\pm 0.316}$ | $1.3545_{\pm 3.846}$ | $1.4197_{\pm 2.092}$ |
| CuTS [62] | $-0.0011_{\pm 0.069}$ | $3.9002_{\pm 1.698}$ | $0.1225_{\pm 0.036}$ | $0.6666_{\pm 0.230}$ | $1.0000_{\pm 0.0}$ | $0.4837_{\pm 0.263}$ | $1.0000_{\pm 0.0}$ | $0.8782_{\pm 0.503}$ | $0.9881_{\pm 0.021}$ | $0.3538_{\pm 0.158}$ | $0.0730_{\pm 0.030}$ | $0.2478_{\pm 0.117}$ | $0.2113_{\pm 0.230}$ | $52.9381_{\pm 221.286}$ | $0.0080_{\pm 0.009}$ | $0.4946_{\pm 0.399}$ | $0.1983_{\pm 0.114}$ | $0.8083_{\pm 0.737}$ | $3.4122_{\pm 7.741}$ | $1.6919_{\pm 2.045}$ |
| DDPM [28] | $-0.0177_{\pm 0.067}$ | $4.8887_{\pm 1.987}$ | $0.1193_{\pm 0.043}$ | $0.7684_{\pm 0.209}$ | $1.0000_{\pm 0.0}$ | $0.6432_{\pm 0.410}$ | $1.0000_{\pm 0.0}$ | $0.6369_{\pm 0.385}$ | $0.9891_{\pm 0.021}$ | $0.3487_{\pm 0.192}$ | $0.0784_{\pm 0.046}$ | $0.2100_{\pm 0.119}$ | $0.1210_{\pm 0.105}$ | $87.0285_{\pm 435.788}$ | $0.0057_{\pm 0.007}$ | $0.3521_{\pm 0.297}$ | $0.1727_{\pm 0.096}$ | $0.5576_{\pm 0.391}$ | $2.1915_{\pm 4.587}$ | $1.5801_{\pm 2.234}$ |
| FSMOTE [59] | $0.0107_{\pm 0.054}$ | $5.2332_{\pm 2.276}$ | $0.1004_{\pm 0.044}$ | $0.7711_{\pm 0.219}$ | $1.0000_{\pm 0.0}$ | $0.6044_{\pm 0.254}$ | $1.0000_{\pm 0.0}$ | $0.6476_{\pm 0.441}$ | $0.9865_{\pm 0.021}$ | $0.3166_{\pm 0.171}$ | $0.0740_{\pm 0.041}$ | $0.1907_{\pm 0.103}$ | $0.0991_{\pm 0.081}$ | $101.7921_{\pm 516.083}$ | $0.0065_{\pm 0.007}$ | $0.3200_{\pm 0.230}$ | $0.1685_{\pm 0.079}$ | $0.4577_{\pm 0.324}$ | $1.4852_{\pm 3.806}$ | $1.4906_{\pm 2.151}$ |
| GOGGLE [41] | $0.0624_{\pm 0.121}$ | $3.4861_{\pm 1.464}$ | $0.1097_{\pm 0.038}$ | $0.6574_{\pm 0.242}$ | $1.0000_{\pm 0.0}$ | $0.4513_{\pm 0.273}$ | $1.0000_{\pm 0.0}$ | $0.8268_{\pm 0.511}$ | $0.9921_{\pm 0.015}$ | $0.5668_{\pm 0.325}$ | $0.1007_{\pm 0.052}$ | $0.3848_{\pm 0.116}$ | $0.5394_{\pm 0.512}$ | $175.0811_{\pm 857.559}$ | $0.0114_{\pm 0.009}$ | $0.6391_{\pm 0.440}$ | $0.2339_{\pm 0.113}$ | $0.7762_{\pm 0.581}$ | $2.5000_{\pm 5.170}$ | $1.9780_{\pm 2.227}$ |
| PreFair [49] | $-0.0163_{\pm 0.059}$ | $4.3822_{\pm 1.918}$ | $0.1253_{\pm 0.040}$ | $0.6760_{\pm 0.231}$ | $1.0000_{\pm 0.0}$ | $0.4861_{\pm 0.258}$ | $1.0000_{\pm 0.0}$ | $0.8658_{\pm 0.524}$ | $0.9861_{\pm 0.022}$ | $0.3228_{\pm 0.162}$ | $0.0704_{\pm 0.034}$ | $0.2151_{\pm 0.098}$ | $0.1482_{\pm 0.186}$ | $56.1625_{\pm 237.270}$ | $0.0068_{\pm 0.008}$ | $0.4420_{\pm 0.378}$ | $0.1858_{\pm 0.111}$ | $0.7437_{\pm 0.712}$ | $3.4116_{\pm 7.906}$ | $1.6751_{\pm 2.040}$ |
| SMOTE [15] | $0.0503_{\pm 0.077}$ | $4.9191_{\pm 3.232}$ | $0.0825_{\pm 0.032}$ | $0.8278_{\pm 0.174}$ | $1.0000_{\pm 0.0}$ | $0.8144_{\pm 0.230}$ | $1.0000_{\pm 0.0}$ | $0.5720_{\pm 0.440}$ | $0.9895_{\pm 0.021}$ | $0.2604_{\pm 0.171}$ | $0.0590_{\pm 0.038}$ | $0.1916_{\pm 0.110}$ | $0.1137_{\pm 0.108}$ | $96.5422_{\pm 488.294}$ | $0.0082_{\pm 0.007}$ | $0.3488_{\pm 0.284}$ | $0.1738_{\pm 0.093}$ | $0.5138_{\pm 0.374}$ | $1.5052_{\pm 4.457}$ | $1.5856_{\pm 1.960}$ |
| SMOTE-CDNN [64] | $0.0480_{\pm 0.075}$ | $4.9460_{\pm 3.234}$ | $0.0832_{\pm 0.032}$ | $0.8280_{\pm 0.173}$ | $1.0000_{\pm 0.0}$ | $0.8103_{\pm 0.228}$ | $1.0000_{\pm 0.0}$ | $0.5721_{\pm 0.438}$ | $0.9898_{\pm 0.020}$ | $0.2588_{\pm 0.168}$ | $0.0586_{\pm 0.037}$ | $0.1915_{\pm 0.111}$ | $0.1137_{\pm 0.109}$ | $88.4434_{\pm 444.627}$ | $0.0080_{\pm 0.007}$ | $0.3462_{\pm 0.280}$ | $0.1723_{\pm 0.092}$ | $0.5106_{\pm 0.374}$ | $1.5133_{\pm 4.483}$ | $1.5469_{\pm 1.984}$ |
| Tabfairgan [51] | $0.0553_{\pm 0.093}$ | $4.4360_{\pm 2.173}$ | $0.0903_{\pm 0.033}$ | $0.7543_{\pm 0.217}$ | $1.0000_{\pm 0.0}$ | $0.6311_{\pm 0.391}$ | $1.0000_{\pm 0.0}$ | $0.6645_{\pm 0.416}$ | $0.9869_{\pm 0.022}$ | $0.3605_{\pm 0.226}$ | $0.0811_{\pm 0.051}$ | $0.2168_{\pm 0.138}$ | $0.1427_{\pm 0.144}$ | $154.9619_{\pm 768.136}$ | $0.0103_{\pm 0.011}$ | $0.4217_{\pm 0.338}$ | $0.1829_{\pm 0.079}$ | $0.5604_{\pm 0.330}$ | $1.6614_{\pm 3.817}$ | $1.8995_{\pm 2.476}$ |
| TabularARGN [61] | $0.0194_{\pm 0.065}$ | $5.6073_{\pm 3.104}$ | $0.0867_{\pm 0.032}$ | $0.7775_{\pm 0.204}$ | $1.0000_{\pm 0.0}$ | $0.7189_{\pm 0.369}$ | $1.0000_{\pm 0.0}$ | $0.6387_{\pm 0.373}$ | $0.9851_{\pm 0.022}$ | $0.2823_{\pm 0.169}$ | $0.0675_{\pm 0.042}$ | $0.1752_{\pm 0.101}$ | $0.0903_{\pm 0.084}$ | $109.4713_{\pm 558.539}$ | $0.0057_{\pm 0.007}$ | $0.2704_{\pm 0.212}$ | $0.1569_{\pm 0.065}$ | $0.4385_{\pm 0.314}$ | $1.3745_{\pm 3.807}$ | $1.6761_{\pm 2.395}$ |
| TVAE [65] | $0.0579_{\pm 0.087}$ | $5.0782_{\pm 2.283}$ | $0.0722_{\pm 0.026}$ | $0.8267_{\pm 0.169}$ | $1.0000_{\pm 0.0}$ | $0.9006_{\pm 0.421}$ | $1.0000_{\pm 0.0}$ | $0.5510_{\pm 0.365}$ | $0.9981_{\pm 0.003}$ | $0.2803_{\pm 0.141}$ | $0.0605_{\pm 0.030}$ | $0.2617_{\pm 0.088}$ | $0.1049_{\pm 0.101}$ | $114.2559_{\pm 579.079}$ | $0.0064_{\pm 0.007}$ | $0.2672_{\pm 0.187}$ | $0.1637_{\pm 0.072}$ | $0.4288_{\pm 0.308}$ | $1.3660_{\pm 3.782}$ | $1.7536_{\pm 2.428}$ |
| SyRFDRes $\phi = 2/rlx$ | $0.0196_{\pm 0.059}$ | $5.8742_{\pm 3.127}$ | $0.0854_{\pm 0.037}$ | $0.8499_{\pm 0.152}$ | $1.0000_{\pm 0.0}$ | $0.8949_{\pm 0.228}$ | $1.0000_{\pm 0.0}$ | $0.4396_{\pm 0.257}$ | $0.9891_{\pm 0.021}$ | $0.2591_{\pm 0.157}$ | $0.0626_{\pm 0.039}$ | $0.1855_{\pm 0.111}$ | $0.0776_{\pm 0.076}$ | $87.8555_{\pm 430.760}$ | $0.0051_{\pm 0.007}$ | $0.2220_{\pm 0.193}$ | $0.1357_{\pm 0.054}$ | $0.4136_{\pm 0.315}$ | $1.3091_{\pm 3.806}$ | $1.5861_{\pm 2.420}$ |
| SyRFDRes $\phi = 2/str$ | $0.0189_{\pm 0.060}$ | $5.8918_{\pm 3.291}$ | $0.0857_{\pm 0.037}$ | $0.8490_{\pm 0.150}$ | $1.0000_{\pm 0.0}$ | $0.9103_{\pm 0.211}$ | $1.0000_{\pm 0.0}$ | $0.4409_{\pm 0.257}$ | $0.9898_{\pm 0.020}$ | $0.2557_{\pm 0.153}$ | $0.0617_{\pm 0.038}$ | $0.1861_{\pm 0.113}$ | $0.0777_{\pm 0.076}$ | $91.0870_{\pm 464.848}$ | $0.0051_{\pm 0.007}$ | $0.2223_{\pm 0.193}$ | $0.1339_{\pm 0.053}$ | $0.4150_{\pm 0.316}$ | $1.3096_{\pm 3.806}$ | $1.5704_{\pm 2.425}$ |
| SyRFDRes $\phi = 4/rlx$ | $0.0168_{\pm 0.060}$ | $6.0501_{\pm 3.510}$ | $0.0863_{\pm 0.037}$ | $0.8534_{\pm 0.149}$ | $1.0000_{\pm 0.0}$ | $0.9235_{\pm 0.198}$ | $1.0000_{\pm 0.0}$ | $0.4362_{\pm 0.255}$ | $0.9898_{\pm 0.020}$ | $0.2508_{\pm 0.155}$ | $0.0606_{\pm 0.038}$ | $0.1815_{\pm 0.112}$ | $0.0763_{\pm 0.076}$ | $94.4684_{\pm 474.502}$ | $0.0049_{\pm 0.006}$ | $0.2199_{\pm 0.202}$ | $0.1314_{\pm 0.055}$ | $0.4097_{\pm 0.319}$ | $1.3041_{\pm 3.805}$ | $1.5476_{\pm 2.425}$ |
| SyRFDRes $\phi = 4/str$ | $0.0139_{\pm 0.061}$ | $6.1532_{\pm 3.424}$ | $0.0878_{\pm 0.040}$ | $0.8523_{\pm 0.150}$ | $1.0000_{\pm 0.0}$ | $0.9102_{\pm 0.198}$ | $1.0000_{\pm 0.0}$ | $0.4373_{\pm 0.254}$ | $0.9900_{\pm 0.020}$ | $0.2472_{\pm 0.150}$ | $0.0600_{\pm 0.037}$ | $0.1815_{\pm 0.112}$ | $0.0747_{\pm 0.076}$ | $52.9416_{\pm 259.250}$ | $0.0048_{\pm 0.006}$ | $0.2190_{\pm 0.200}$ | $0.1293_{\pm 0.054}$ | $0.4071_{\pm 0.317}$ | $1.3105_{\pm 3.805}$ | $1.5378_{\pm 2.431}$ |
| SyRFDRes $\phi = 8/rlx$ | $0.0121_{\pm 0.059}$ | $6.0434_{\pm 3.258}$ | $0.0883_{\pm 0.039}$ | $0.8523_{\pm 0.149}$ | $1.0000_{\pm 0.0}$ | $0.9092_{\pm 0.205}$ | $1.0000_{\pm 0.0}$ | $0.4335_{\pm 0.250}$ | $0.9895_{\pm 0.020}$ | $0.2499_{\pm 0.153}$ | $0.0605_{\pm 0.038}$ | $0.1812_{\pm 0.105}$ | $0.0759_{\pm 0.075}$ | $79.8875_{\pm 399.623}$ | $0.0049_{\pm 0.006}$ | $0.2164_{\pm 0.193}$ | $0.1285_{\pm 0.051}$ | $0.4051_{\pm 0.316}$ | $1.3116_{\pm 3.805}$ | $1.5479_{\pm 2.432}$ |
| SyRFDRes $\phi = 8/str$ | $0.0137_{\pm 0.061}$ | $5.8961_{\pm 2.959}$ | $0.0883_{\pm 0.039}$ | $0.8518_{\pm 0.149}$ | $1.0000_{\pm 0.0}$ | $0.9082_{\pm 0.194}$ | $1.0000_{\pm 0.0}$ | $0.4346_{\pm 0.254}$ | $0.9897_{\pm 0.020}$ | $0.2483_{\pm 0.151}$ | $0.0601_{\pm 0.037}$ | $0.1836_{\pm 0.110}$ | $0.0762_{\pm 0.078}$ | $51.0275_{\pm 247.001}$ | $0.0050_{\pm 0.007}$ | $0.2236_{\pm 0.203}$ | $0.1329_{\pm 0.053}$ | $0.4082_{\pm 0.314}$ | $1.3081_{\pm 3.806}$ | $1.4725_{\pm 2.413}$ |
| SyRFDRes var/*rlx* | $0.0895_{\pm 0.111}$ | $4.4273_{\pm 2.837}$ | $0.0835_{\pm 0.034}$ | $0.7904_{\pm 0.231}$ | $1.0000_{\pm 0.0}$ | $0.7021_{\pm 0.334}$ | $1.0000_{\pm 0.0}$ | $0.6551_{\pm 0.509}$ | $0.9886_{\pm 0.022}$ | $0.3397_{\pm 0.245}$ | $0.0733_{\pm 0.047}$ | $0.2316_{\pm 0.136}$ | $0.2051_{\pm 0.216}$ | $152.7458_{\pm 772.619}$ | $0.0129_{\pm 0.008}$ | $0.5018_{\pm 0.488}$ | $0.2028_{\pm 0.114}$ | $0.5931_{\pm 0.492}$ | $1.7786_{\pm 5.415}$ | $1.8887_{\pm 2.320}$ |
| SyRFDRes var/*str* | $0.1049_{\pm 0.126}$ | $4.2615_{\pm 2.848}$ | $0.0815_{\pm 0.035}$ | $0.7830_{\pm 0.232}$ | $1.0000_{\pm 0.0}$ | $0.6972_{\pm 0.347}$ | $1.0000_{\pm 0.0}$ | $0.6592_{\pm 0.507}$ | $0.9870_{\pm 0.021}$ | $0.3433_{\pm 0.252}$ | $0.0735_{\pm 0.048}$ | $0.2417_{\pm 0.156}$ | $0.2234_{\pm 0.238}$ | $156.5711_{\pm 792.395}$ | $0.0144_{\pm 0.009}$ | $0.5474_{\pm 0.522}$ | $0.2065_{\pm 0.115}$ | $0.6239_{\pm 0.508}$ | $1.7743_{\pm 5.416}$ | $1.9692_{\pm 2.295}$ |

**Table 2: Mean and std of all diversity and fidelity metrics across all datasets (↑/↓: higher/lower is better; $\overset{0}{\rightarrow}$: best near zero; no arrow: best at intermediate values). Green and red mark the three best and worst values per column, with darker shades for higher rankings.**

attribute, while $\text{WD}_{\text{PCA}}$ evaluates the same distance after projecting the data onto the first three principal PCA components. As we can see, GOGGLE and CasTGAN achieve the worst WD of around 175.08 and 141.03, respectively, indicating that generated tuples suffer from substantial distributional shifts with respect to the real minority class. Among baselines, CuTS achieves the best WD of 52.94, followed by PreFair of 56.16, while for $\text{WD}_{\text{PCA}}$, TabularARGN obtains the best value of 0.157, followed by DDPM and SMOTE-CDNN at around 0.172. Nevertheless, SyRFDRes achieves better performance than all baselines on $\text{WD}_{\text{PCA}}$, with all fixed-threshold configurations between 0.128 and 0.136, showing the effectiveness of involving RFDs in the augmentation approach. Moreover, on the WD score, SyRFDRes with threshold 8 on the strict configuration obtains the best value around 51.03.

*pMSE and MMD.* pMSE evaluates how easily a classifier can distinguish real from generated tuples; values close to zero indicate that the two datasets are indistinguishable. MMD measures the distance between real and generated joint distributions by comparing their kernel mean embeddings, capturing discrepancies in distributional patterns beyond individual moments. A value of zero indicates identical distributions. As we can see, the lowest worst have been achieved from the data generated with GOGGLE, CasTGAN, and TabFairGAN, with pMSE above 0.010 and MMD above 0.420. Among baselines, CTAB-GAN+ achieves the best pMSE of 0.0056 and TabularARGN the best MMD of 0.270, followed by TVAE with MMD of 0.267. SyRFDRes outperforms all baselines on both metrics, with fixed-threshold configurations that achieve pMSE between 0.0048 and 0.0051 and MMD between 0.216 and 0.224.

*Moment differences.* $\Delta_\mu$, $\Delta_v$, and $\Delta_{\text{skew}}$ evaluate mismatch on location, spread, and asymmetry, respectively, achieving a value equal to zero for perfectly matching distributions. As we can see, GOGGLE and CasTGAN are consistently the weakest across all three moments, with $\Delta_\mu$ above 0.65 and $\Delta_{\text{skew}}$ above 1.90. Among baselines, SMOTE and SMOTE-CDNN achieve $\Delta_\mu$ of 0.514 and 0.511, while TabularARGN records the best $\Delta_v$ of 1.375 and CTAB-GAN+ the best $\Delta_{\text{skew}}$ of 1.420. SyRFDRes achieves better performance than all baselines on $\Delta_\mu$ and $\Delta_v$ when configured with fixed-threshold, while for $\Delta_{\text{skew}}$ it is among the best performers.

*Overall results.* Across fidelity metrics, fixed-threshold configurations of SyRFDRes outperform baselines in most cases, with the largest gains on MMD and $\text{WD}_{\text{PCA}}$, which capture inter-attribute structure beyond marginal distributions. This is likely because the RFD-driven generation process constrains attribute values to remain consistent with the relational structure of the real minority class. Unlike standard generative models, which typically match the overall data distribution without modeling relational constraints, RFD validation combined with the proposed loss guides generation toward tuples satisfying minority-class relationships, preserving inter-attribute correlations that other approaches could violate.

## 5.2 Diversity results

*Recall and Coverage.* Recall measures the fraction of real minority tuples covered by the neighborhood of at least one generated tuple, with neighborhoods defined by the generated set, while coverage uses the same criterion with neighborhoods defined by the real set instead. Both metrics equal 1.00 for all algorithms, indicating that all generators produce tuples that significantly enrich the real minority sample.

*Precision and Density.* Precision measures the fraction of generated tuples that fall inside at least one real neighborhood, penalizing out-of-distribution generation, while Density refines this by counting how many real neighborhoods each generated tuple falls into, further penalizing generators that concentrate values in a few dense regions. As we can see from the results, the worst scores have been achieved by GOGGLE, PreFair, and CuTS, with Precision below 0.676 and Density below 0.484. Among the other baselines, SMOTE, SMOTE-CDNN, and TVAE achieve the highest values for the Precision, with TVAE also achieving the Density score of 0.901. Regarding SyRFDRes, it achieves better performance than all baselines on both metrics. As we can see, for most fixed-threshold configurations, it achieves a precision between 0.849 and 0.853 and a density between 0.895 and 0.924.

*Silhouette score and DBI.* The Silhouette score, computed by treating real and generated samples as two clusters, measures their separability: values close to zero indicate stronger overlap. The

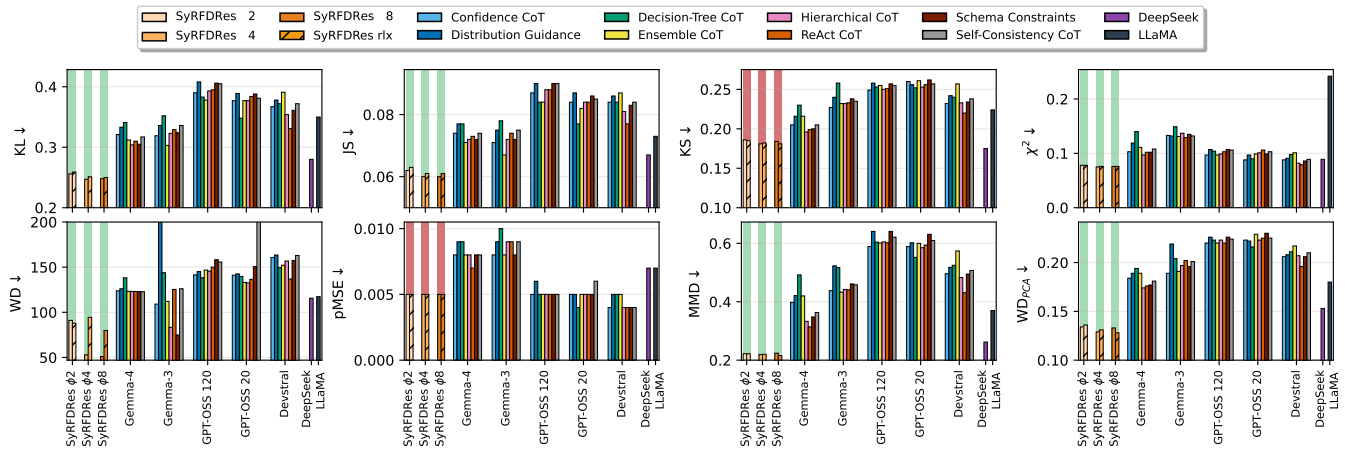



**Figure 1: Comparison of mean fidelity metrics between data generated by SyRFDRes and those with LLM-based methods. The red background ● highlights that data generated by SyRFDRes achieves worse values compared to those generated by LLMs, while the green background ● highlights the opposite.**

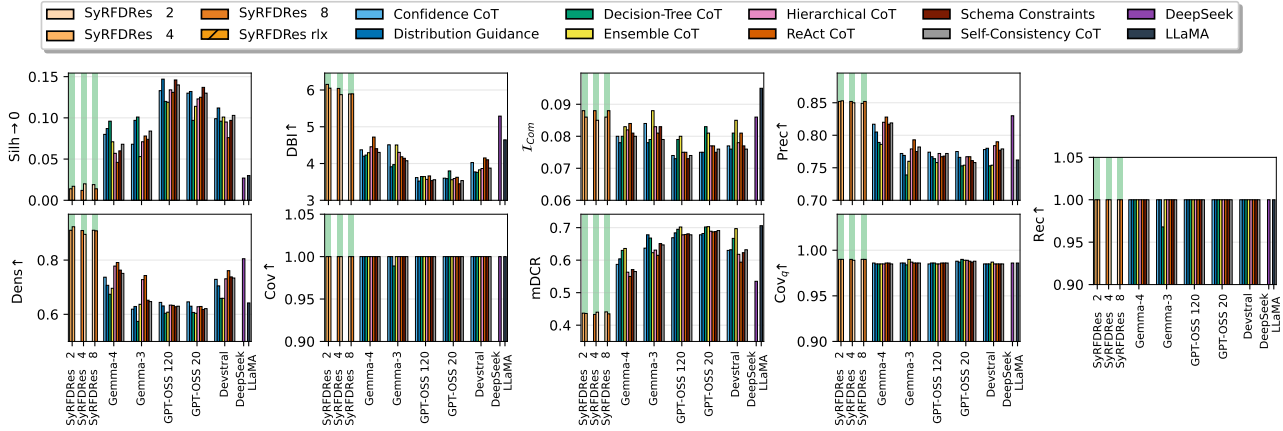



**Figure 2: Comparison of mean diversity metrics between data generated by SyRFDRes and those with LLM-based methods.**

DBI complements this measure by evaluating the ratio between within-set dispersion and centroid separation, where higher values indicate greater similarity between the real minority samples and the generated tuples. The worst scores have been obtained by GOGGLE and CasTGAN, with Silhouette above 0.062 and DBI below 3.90, indicating that their generated tuples form a cluster spatially separated from the real minority samples. Among baselines, CuTS achieves the best Silhouette of −0.0011, but not a high value for DBI. Instead, FSMOTE and TabularARGN achieved good DBIs of 5.2332 and 5.6073, respectively. SyRFDRes achieves better performance than all baselines on the DBI when executed with RFDs extracted with fixed-threshold and a good Silhouette of 0.0121.

*Intra-compactness and mean distance to closest record.* $\mathcal{I}_{Com}$ measures internal diversity of the generated set via the average attribute variance. Values close to zero indicate mode collapse, while values close to the real minority variance suggest a realistic spread. mDCR measures the average distance between each generated tuple and its nearest real counterpart: very small values indicate duplication of existing records, whereas very large values suggest that generated samples move away from the real data distribution. Therefore, intermediate mDCR values are desirable. All methods record $\mathcal{I}_{Com}$ between 0.072 and 0.125, well above zero, which rules out mode collapse, with no method standing out. On mDCR, SMOTE, SMOTE-CDNN, and TVAE achieve the lowest values (around 0.55–0.57), suggesting slight over-proximity to real samples. CuTS and PreFair achieve the highest mDCR values, indicating tuples less similar to the real minority samples, potentially harming fidelity scores. SyRFDRes fixed-threshold configurations achieve intermediate mDCR values between 0.434 and 0.441, indicating that the generated tuples are novel with respect to the real minority samples while remaining well within their distributional support.

*Quantile-bin coverage.* $\text{Cov}_q$ measures the fraction of real decile bins populated by at least one generated value, verifying that the generator covers the full range of each attribute, including low-density tails. A value of 1 indicates complete coverage. All algorithms achieve high $\text{Cov}_q$ values between 0.985 and 0.998, while

SyRFDRes with fixed-threshold configurations records values between 0.989 and 0.990, near the best baselines.

*Overall results.* SyRFDRes executed with $\text{RFD}_c$s extracted from fixed-threshold discovery achieves better performance with respect to the best baselines on most diversity metrics, with the most notable improvements on Precision, Density, and DBI. These results show that the RFD-driven generation produces tuples that are similarly distributed to the real minority samples and that these cover the minority manifold densely without out-of-distribution drift.

## 5.3 Fixed and automatically inferred thresholds

As we have seen, for both diversity and fidelity metrics, SyRFDRes executed with $\text{RFD}_c$s with automatically inferred thresholds exhibits weaker diversity scores. This degradation can be due to the fact that with automatically inferred thresholds, the discovery process can produce multiple valid $\text{RFD}_c$s involving the same attributes but with different levels of granularity, e.g., $\{A_2, B_1 \rightarrow C_1\}$ and $\{A_1, B_4 \rightarrow C_3\}$. Although each dependency is valid and minimal, their coexistence can define a more complex constraint space, as the generated tuples must simultaneously satisfy all the extracted $\text{RFD}_c$s. This can introduce heterogeneous and difficult-to-reconcile constraints, reducing the effectiveness of the guidance provided to the generator. Moreover, because the discovery algorithm initially infers thresholds on small groups of samples and subsequently validates the dependencies on the entire minority class, the resulting RFDs may reflect patterns present in subgroups of tuples. Even when validated globally, these may not represent the most robust and significant relationships in the entire minority distribution.

## 5.4 Comparison of SyRFDRes against LLM generated data

Figures 1 and 2 compare SyRFDRes against the full set of LLM-based generators across fidelity and diversity metrics, respectively. Regarding fidelity metrics, as visible in Figure 2a, data generated by SyRFDRes on both strict and relaxed modalities achieves better scores than those generated with LLM-based strategies on most metrics (↓). The gap is particularly pronounced on KL, JS, $\chi^2$, WD, MMD, and $\text{WD}_{\text{PCA}}$, where even the LLM strategies that generate the highest-quality data overall, i.e., DeepSeek and LLaMA with zero-shot prompting, record substantially higher values than SyRFDRes. However, on KS and pMSE metrics, several LLM-based strategies achieve slightly better average values than SyRFDRes, as highlighted by the red background. On KS, this suggests that certain LLM strategies, particularly those with explicit distribution guidance, can match the per-attribute cumulative structure of the minority class closely enough to outperform SyRFDRes on the worst-case CDF gap, even though they fail to preserve joint distributional properties captured by MMD and $\text{WD}_{\text{PCA}}$. On pMSE, the advantage of some LLM strategies indicates that their generated tuples are harder to separate from real minority samples by a linear classifier, possibly because zero-shot generation tends to produce values close to the observed minority mean, reducing linear discriminability at the cost of lower diversity.

Regarding diversity metrics, as we can see from Figure 2b, LLM-based strategies achieved scores almost always worse than those of SyRFDRes. On the Silhouette, data generated by LLM-based strategies record substantially higher values than SyRFDRes, with GPT-OSS models and Devstral exceeding 0.10 across most prompting strategies, showing that data generated with LLMs form a cluster with values that are more separated from the real minority distribution. The gap is consistent across all eight prompting strategies, except for zero-shot prompting approaches that show slightly worse performance than SyRFDRes. Similarly, for the DBI metric, most LLM settings achieve a DBI score that does not exceed 4.5 for most prompting approaches, and several drop below 4.0, indicating weak overlap between the generated and real sets. Structured strategies such as Decision-Tree CoT and Ensemble CoT tend to achieve marginally higher DBI values within the LLM group, but remain far below SyRFDRes. Among LLM strategies, Distribution Guidance and Self-Consistency CoT tend to achieve slightly higher Precision, suggesting that explicitly conditioning generation on observed statistics reduces out-of-distribution drift at the margin. Regarding Recall and Coverage, all LLM strategies achieve values at or near 1.00, except for the data generated with Decision-Tree CoT with the Gemma-4 model, which achieves a recall value of about 0.98. About mDCR, LLM strategies record higher values than SyRFDRes, ranging from 0.55 to 0.65 for most approaches, indicating that LLM-generated tuples tend to drift further from the real minority support. The only exception is data generated by DeepSeek, achieving mDCR closer to SyRFDRes. On $\mathcal{I}_{Com}$ and $\text{Cov}_q$, LLM strategies perform comparably to SyRFDRes, with no approach showing systematic mode collapse.

# 6 UTILITY EVALUATION

Figures 3, 4, and 5 provide an overview of the evaluation of augmented datasets when trained on 13 classifiers of various types. These results are compared against a baseline using non-augmented datasets and 63 versions of augmented datasets featuring configurations based on GANs and LLMs, such as classical oversampling techniques, GAN-based generators and diffusion models, fairness-aware generators, and five LLMs executed using the eight prompting strategies discussed above.

*Accuracy (Fig. 3).* Although all datasets are imbalanced, the accuracy achieved on the baseline dataset ranges from 0.91 (Ridge, Logistic Regression) to 0.95 (LGBM, XGBoost). As we can see, most techniques match or marginally exceed these values, and SyRFDRes reaches up to 1.00 for LGBM, ExtraTrees, and XGBoost, on par with the strongest GAN-based and LLM-based competitors. Moreover, as we can see for the three linear classifiers considered, i.e., LinearSVC, Logistic Regression, and Ridge, accuracy decreases with almost every augmentation technique, and SyRFDRes achieves metrics ranging from 59 to 62 out of the 63 competing configurations, depending on the classifier. This is probably due to the fact that a linear model trained on the original dataset can already achieve a deceptively high accuracy by essentially always predicting the majority class. Once trained on a balanced dataset, the same model starts correctly classifying minority-class instances as well, which occasionally leads to misclassifications and therefore a marginal drop in accuracy scores.

*F1-score (Fig. 4).* On the benchmark dataset, the F1 score drops for boosting classifiers that are not optimized for class imbalance

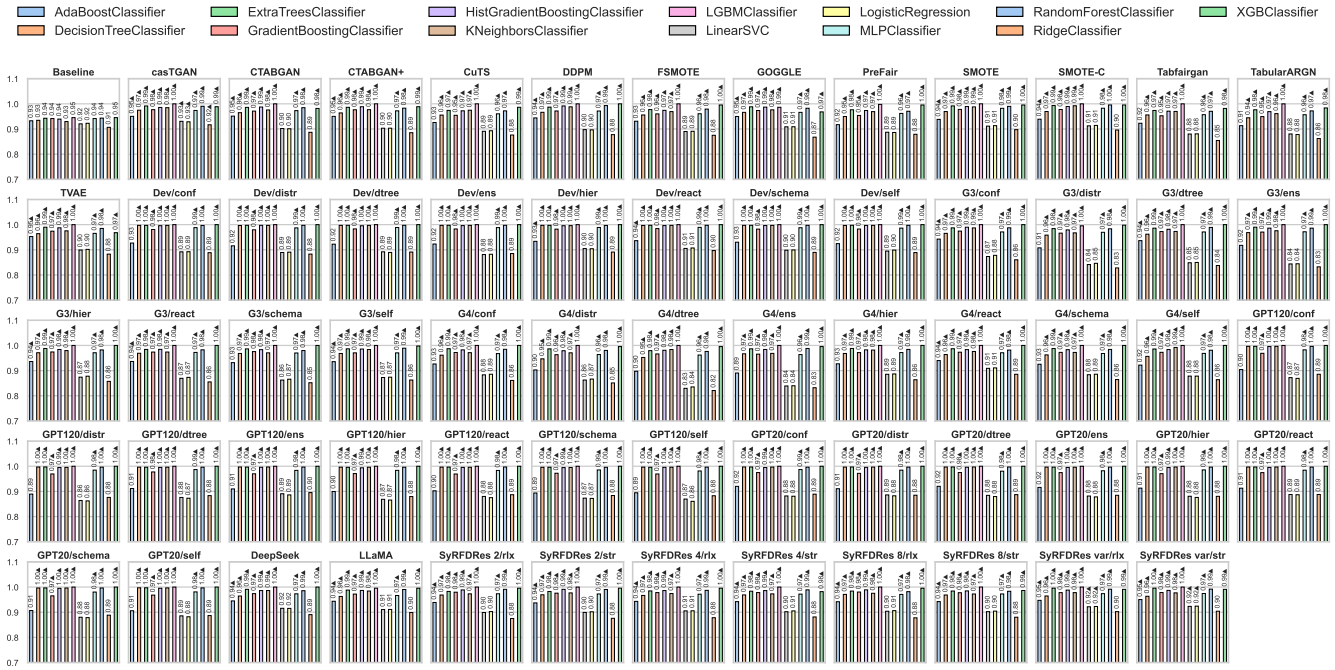



**Figure 3: Comparison of accuracy values achieved by 13 classifiers on all datasets augmented with GAN-based, LLM-based, and SyRFDRes techniques against the baseline dataset. The black upward-pointing triangle (▲) above a bar indicates that the model's average accuracy exceeds the baseline's average accuracy for that same classifier.**

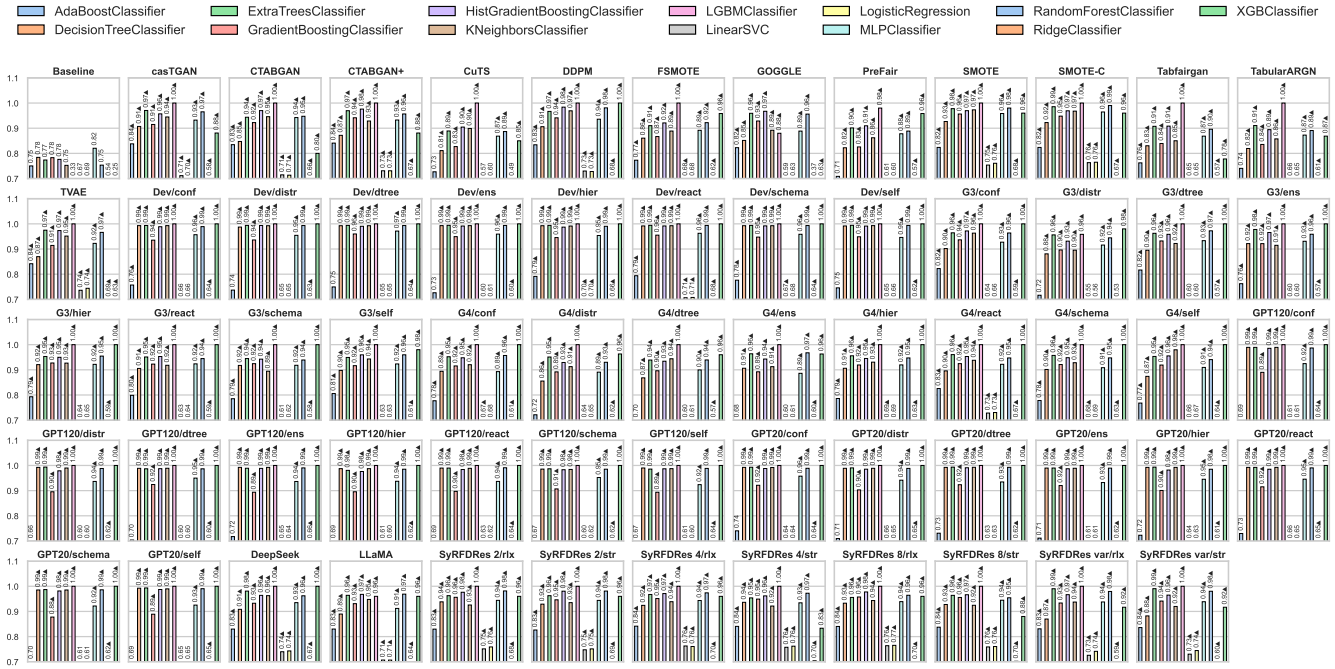



**Figure 4: Comparison of F1 values achieved by 13 classifiers on all datasets augmented with GAN-based, LLM-based, and SyRFDRes techniques against the baseline dataset.**

(0.33 for LGBM, 0.25 for XGBoost) and remains moderate for the three linear classifiers (0.54 for Ridge, 0.67 for LinearSVC, 0.69 for logistic regression). As can be seen, each family of boosting techniques bridges this gap for LGBM and XGBoost, but SyRFDRes




**Figure 5: Comparison of average precision and recall for each augmentation technique versus the baseline. Reported values are *speedups*: the ratio between the precision (or recall) of a model trained on data from a given augmentation method and that of the same model trained on the baseline dataset. A value of 2 means the augmented model reaches twice the baseline precision (or recall); a value of 0.5 means worse performance. The dashed red horizontal line at $y = 1$ marks the baseline: markers above it indicate improvement, those below indicate degradation.**

is the only technique that improves upon the baseline for all 13 classifiers across all eight of its configurations. For the three linear classifiers, however, Fig. 4 reveals a much less uniform picture, in which whether a technique improves or degrades the F1 score relative to the baseline depends heavily on the family of augmentation techniques. Moreover, LLM-based configurations are the most likely to compromise these classifiers, regardless of the underlying model or prompting strategy: 79% of them (33/42) drop below the baseline F1 score for LinearSVC, and 83% (35/42) do so for logistic regression (e.g., `G3/react`, `GPT120/conf`, among many others). GAN-based approaches generally achieve superior performance; nevertheless, they tend to be unstable, with roughly half of them underperforming the baseline for these two classifiers (including GOGGLE, CuTS, PreFair, Tabfairgan, and TabularARGN, among others). SyRFDRes, in contrast, is consistently reliable: all eight of its configurations outperform the baseline for both classifiers, achieving scores of up to 0.76 (LinearSVC) and 0.77 (logistic regression). For Ridge, the situation is mixed, with only three models outperforming the baseline.

*Precision and Recall (Fig. 5).* Reporting the ratio to the baseline per classifier and technique makes relative gains directly comparable across the grid: markers above the dashed $y = 1$ line denote an improvement in precision or recall over the baseline, markers below denote a degradation. As we can see, SyRFDRes's eight configurations place both the precision and the recall marker at or above 1 for every classifier, with the largest speedups, up to 6×, observed for LGBM and XGBoost, consistent with their very low baseline precision and recall discussed above. Competing techniques are markedly less consistent: for the three linear classifiers, several LLM-based configurations place one or both markers below the

$y = 1$ line, whereas SyRFDRes's markers remain at or above the line for every classifier throughout all eight configurations, with one exception: for the Ridge classifier, the variable-threshold configurations (`var/rlx`, `var/str`) achieve a more modest speedup than the fixed-threshold ones, consistent with the less score of F1 improvement reported above, though both ratios remain above 1.

# 7 FULL ABLATION RESULTS

Table 3 details the complete numerical results of the ablation study described in Sec. 6 of the main paper. For each configuration and operating mode, we report the mean across 35 benchmark datasets of distributional fidelity (i.e., SWD), sample diversity (i.e., Coverage), constraint compliance (i.e., RFD Violation Rate), acceptance rate of the generator, average number of sampling rounds, and total training and generation times.

As we can see, in relaxed mode, removing the SoftRFD loss reduces the average training time from 32.9s to 165.0s with a small impact on the fidelity loss, because the validator is absent and no gradient signal flows through the constraint penalty. However, this mode does not guarantee strict RFD compliance. Instead, in Strict mode, the SoftRFD loss reduces the mean number of sampling rounds from 1.357 to 1.321. Removing the RFD validator is the only setting that affects compliance, i.e., 1.7% in relaxed and 1.8% in strict mode, and reduces generation time to 0.01s, confirming that much of the generation time is spent in the RFD checking. Concerning the early stopping, removing this check led to an increasing training time of around 232–267s, also yielding slightly improved fidelity.

| Mode | Variant | SWD↓ | Cov.↑ | Viol.↓ | Acc. | Rnd. | Tr.(s) | Ge.(s) |
|---|---|---|---|---|---|---|---|---|
| Relaxed | Full (SyRFDRes) | 0.0578 | 0.6576 | 0.0000 | 0.9693 | 1.000 | 165.0 | 9.39 |
| | w/o MMD Loss | 0.1287 | 0.1978 | 0.0000 | 0.9920 | 1.000 | 138.2 | 8.80 |
| | w/o SoftRFD Loss | 0.0483 | 0.6838 | 0.0000 | 0.9730 | 1.000 | 32.9 | 8.76 |
| | w/o Composite Reg. | 0.0541 | 0.6098 | 0.0000 | 0.9537 | 1.000 | 192.1 | 9.10 |
| | w/o SWD Loss | 0.0366 | 0.6454 | 0.0000 | 0.9800 | 1.000 | 159.9 | 8.82 |
| | w/o Coverage Loss | 0.0975 | 0.5963 | 0.0000 | 0.9497 | 1.000 | 114.0 | 9.46 |
| | w/o Anti-collapse | 0.0489 | 0.6516 | 0.0000 | 0.9849 | 1.000 | 163.1 | 9.64 |
| | w/o RFD Validator | 0.0397 | 0.6550 | **0.0167** | 1.000 | 1.000 | 164.3 | 0.01 |
| | Relaxed Validator | 0.0399 | 0.6550 | 0.0000 | 0.9827 | 1.000 | 163.9 | 9.86 |
| | w/o Rolling Buffer | 0.0399 | 0.6550 | 0.0000 | 0.9827 | 1.000 | 151.2 | 9.77 |
| | w/o Warmup | 0.0800 | 0.6553 | 0.0000 | 0.9870 | 1.000 | 156.8 | 9.50 |
| | w/o ResNet (MLP) | 0.1807 | 0.6291 | 0.0000 | 0.9827 | 1.000 | 153.3 | 9.61 |
| | w/o Early Stopping | 0.0270 | 0.6603 | 0.0000 | 0.9839 | 1.000 | 232.3 | 9.73 |
| Strict | Full (SyRFDRes) | 0.0590 | 0.7709 | 0.0000 | 0.9033 | 1.357 | 183.0 | 5.14 |
| | w/o MMD Loss | 0.1629 | 0.2117 | 0.0000 | 0.9007 | 1.679 | 164.4 | 5.67 |
| | w/o SoftRFD Loss | 0.0625 | 0.7875 | 0.0000 | 0.9176 | 1.321 | 28.8 | 5.56 |
| | w/o Composite Reg. | 0.0673 | 0.7038 | 0.0000 | 0.8332 | 1.393 | 232.6 | 4.46 |
| | w/o SWD Loss | 0.0474 | 0.7489 | 0.0000 | 0.9204 | 1.357 | 188.9 | 5.64 |
| | w/o Coverage Loss | 0.1176 | 0.7000 | 0.0000 | 0.8300 | 1.357 | 133.9 | 4.11 |
| | w/o Anti-collapse | 0.0624 | 0.7546 | 0.0000 | 0.9260 | 1.321 | 191.6 | 5.62 |
| | w/o RFD Validator | 0.0395 | 0.6480 | **0.0178** | 1.000 | 1.000 | 160.6 | 0.01 |
| | Relaxed Validator | 0.0399 | 0.6550 | 0.0000 | 0.9827 | 1.000 | 158.9 | 9.54 |
| | w/o Rolling Buffer | 0.0515 | 0.7564 | 0.0000 | 0.9229 | 1.357 | 177.1 | 5.60 |
| | w/o Warmup | 0.1043 | 0.7583 | 0.0000 | 0.9227 | 1.179 | 176.4 | 5.66 |
| | w/o ResNet (MLP) | 0.2320 | 0.7368 | 0.0000 | 0.9297 | 1.321 | 180.8 | 5.72 |
| | w/o Early Stopping | 0.0355 | 0.7652 | 0.0000 | 0.9193 | 1.357 | 266.6 | 5.68 |

**Table 3: Complete ablation results (mean across 35 datasets). SWD = Sliced Wasserstein Distance (↓); Cov. = Coverage (↑); Viol. = RFD Violation Rate (↓); Acc. = Generator Acceptance Rate; Rnd. = Avg. Sampling Rounds; Tr.(s) = Training time; Ge.(s) = Generation time. *Full* = complete SyRFDRes.**

## 7.1 Full Sampling Diagnostics

Table 4 details the sampling diagnostics of Sec. 6 when considering the generation of different amounts of tuples, i.e., 50, 100, 200, and 500. Results associated with *All configs* in Table 4 represent aggregate results over all 4,384 completed executions of the ablation study, i.e., 2,448 for relaxed mode and 1,936 for strict. As we can see, in the relaxed mode, the acceptance rate remains stable across all values of the number of generated tuples, with an average time ranging between 0.981 and 0.983. In this mode, the generation budget is never exhausted in any of the runs. Concerning the results of the executions in strict mode, the mean acceptance rate decreases slightly with the number of tuples to generate, from 0.937 with 50 tuples to 0.920 with 500. This is consistent with the expectation that finding RFD-compliant tuples becomes harder as more must be produced. Moreover, as we can see, budget exhaustion appears in strict mode at 200 tuples on 1 execution of 28 and returns to zero when generating 500 tuples. Generally speaking, the minimum acceptance rate across all strict runs is 0.011 over all configurations, indicating that only in a few instances does the methodology underlying SyRFDRes require a greater number of generation attempts to produce the new set of RFD-compliant tuples.

## 7.2 Sensitivity Analysis

Figure 6 reports a sensitivity study over 10 key hyperparameters and for each factor. For each hyperparameter, all others are fixed at their default values, and a single parameter is varied over a prescribed range. Each line shows the mean SWD across 35 datasets, and the band indicates the **68%** confidence intervals (CI) on the mean.

As we can see, the results show that SyRFDRes is robust to most hyperparameters over wide ranges. The RFD weight (Fig.6(a)) achieves the largest effect, since increasing its value beyond 10 degrades mean SWD in relaxed mode, suggesting a regularisation of the generator toward constraint satisfaction preserving diversity. Concerning the softmax temperature (Fig.6(b)), it exhibits a U-shaped curve with an optimal range of $[0.25, 0.5]$. This means that overly small values, e.g., 0.25, lead to gradient saturation due to a near-discrete approximation of the constraint violation, whereas excessively high values, e.g., $> 0.5$, overly smooth the penalty, diluting the regulatory guidance needed to enforce RFDs and impairing overall distributional fidelity. Concerning the RFD threshold (Fig.6(c)), results show that a threshold of 4 is a stable operating point. Reducing the threshold, i.e., considering RFDs extracted with threshold 2 for all attributes, results tend to slightly degrade fidelity, especially in the strict mode. Regarding the coverage weight (Fig.6(d)), SWD weight (Fig.6(e)), and the composite regularizer weight (Fig.6(f)), all exhibit flat sensitivity curves within their tested ranges, confirming that the default values are not critical tuning points. Instead, as we can see, results about the variation of learning rate, batch size, and warmup ratio (Figs.6($g$,$h$,$i$)) show similarly monotone results, indicating robustness to standard training hyperparameter choices. Finally, the number of SWD projections (Fig.6(j)) shows a slight quality improvement from 10 to 50 that saturates at 200, showing that the default of 50 provides a good fidelity-cost trade-off.

## 7.3 Runtime Analysis

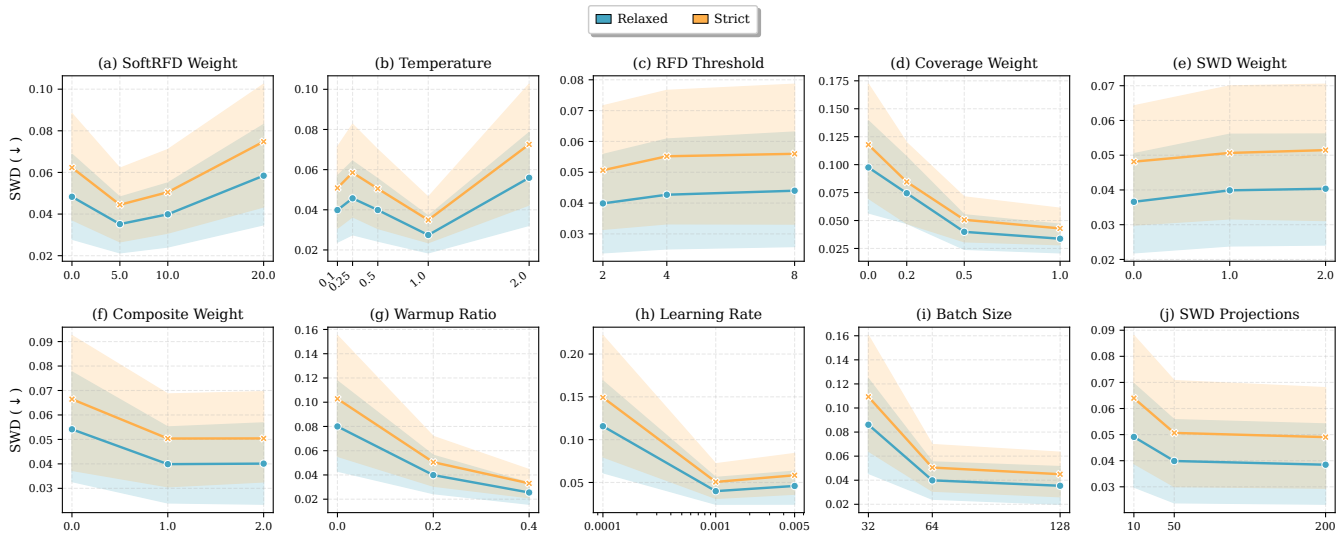



**Figure 6: One-factor-at-a-time (OFAT) sensitivity analysis over 10 hyperparameters (34 configurations). Each curve shows the mean SWD (↓) ± 68% CI across 35 datasets as one hyperparameter varies while all others are fixed at their default values.**

Table 5 provides an overview of the mean execution time for each step of SyRFDRes model across the 35 datasets. Specifically, times have been registered by considering the three non-overlapping steps behind SyRFDRes: *i*) *Warmup*, i.e., unconstrained pre-training without the SoftRFD loss, used to initialise the generator in a meaningful region of the data manifold; *ii*) *Phase 2 (Training)*, i.e., the main constrained training phase with all loss terms active; and *iii*) *Generation*, i.e., synthesis and RFD validation of the synthetic minority tuples after training. As we can see, the times of SyRFDRes executed in strict mode show a small training overhead of about 11% compared to the executions with relaxed mode, probably due to the synchronisation cost of the RFD validator during the training loop. Conversely, strict mode is faster at generation time, showing an average time of 5.1s against the 9.4s of the relaxed mode, probably due to the larger number of tuples on which the validator has to check the RFDs validity. Overall, the total is 188s for the strict mode and 175s for the relaxed, with a small difference of about 8%, demonstrating that the additional constraint imposed in strict mode incurs negligible overall cost.

| Mode | Setting | *n* runs | Exhausted | Acc. Rate | Acc. Min | Rounds |
|---|---|---|---|---|---|---|
| Relaxed | All configs | 2448 | 0/2448 (0.0%) | 0.9792 ± 0.076 | 0.407 | 1.000 |
| Strict | All configs | 1936 | 34/1936 (1.8%) | 0.9202 ± 0.225 | 0.010 | 1.292 |
| Relaxed | 50 | 35 | 0/36 (0.0%) | 0.9834 | 0.629 | 1.000 |
| Relaxed | 100 | 35 | 0/36 (0.0%) | 0.9811 | 0.586 | 1.000 |
| Relaxed | 200 | 35 | 0/36 (0.0%) | 0.9811 | 0.593 | 1.000 |
| Relaxed | 500 | 35 | 0/36 (0.0%) | 0.9810 | 0.615 | 1.000 |
| Strict | 50 | 28 | 0/28 (0.0%) | 0.9367 | 0.106 | 1.000 |
| Strict | 100 | 28 | 0/28 (0.0%) | 0.9398 | 0.041 | 1.143 |
| Strict | 200 | 28 | **1/28 (3.6%)** | 0.9386 | 0.029 | 1.321 |
| Strict | 500 | 28 | 0/28 (0.0%) | 0.9196 | 0.035 | 1.357 |

**Table 4: Sampling diagnostics: budget exhaustion and acceptance rate statistics. Exhausted = portion of runs in which the generation budget was exhausted; Acc. Rate = mean ± std. generator acceptance rate; Acc. Min = minimum acceptance rate observed; Rounds = mean sampling rounds. Rows labelled *All configs* aggregate over all configurations.**

| Mode | Warmup (s) | Phase 2 (s) | Train (s) | Gen. (s) | E2E (s) |
|---|---|---|---|---|---|
| Relaxed | 36.5s | 128.5s | 165.0s | 9.39s | 174.5s |
| Strict | 46.7s | 136.4s | 183.0s | 5.14s | 188.2s |

**Table 5: Runtime analysis for each step of SyRFDRes.**